

Seal: Decentralized Secrets Management

Mysten Labs

June 2026
Version 2.0

Abstract

Blockchain ecosystems provide strong authentication primitives but lack a standardized, blockchain-native model for encrypting data and enforcing access to encrypted content. We present *Seal*, a Decentralized Secrets Management service built on the Sui blockchain that fills this gap by combining Identity-Based Encryption (IBE) with threshold cryptography. In *Seal*, on-chain Move smart contracts define who may decrypt, while a network of independent, off-chain key servers enforces these rules by deriving IBE keys on demand. The core cryptographic construction, Threshold Secret-Shared Boneh-Franklin KEM (TSS-BF-KEM), lets each key server maintain its own master key independently, avoiding distributed key generation, yet enables flexible t -out-of- n threshold decryption so that encryptors choose their own trust assumptions. An accompanying Encrypted BLS signature scheme ensures that derived keys remain private during transport, even in the presence of untrusted aggregators. We prove that the integrated protocol UC-realizes an ideal functionality under the co-BDH assumption in the Random Oracle Model. *Seal*'s policy layer is fully programmable: any condition expressible as a Move smart contract (token ownership, timestamps, on-chain events) can gate decryption, and policies are namespaced per package for composability. We describe *Seal*'s architecture, present example policies, detail the key server and SDK design, present an MPC committee realization with key rotation, and provide complete security proofs.

1 Introduction

Blockchains have fundamentally transformed how we think about consensus and data availability, enabling a new paradigm of decentralized applications that operate without trusted intermediaries. Today's blockchain ecosystems support a diverse range of authentication mechanisms, from traditional seed phrases and hardware wallets to modern solutions like zkLogin and Passkeys, providing users with unprecedented control over their digital identities.

However, despite this evolution in authentication, blockchain systems lack a standardized, blockchain-native model for encrypting data and enforcing access to encrypted content. While authentication answers the question of “who you are”, encryption addresses the equally critical question of “what you are allowed to decrypt, and under what condition”. This gap creates significant limitations for applications that require secure, policy-driven access to sensitive data in decentralized environments.

The need for blockchain-native encryption spans a broad spectrum of use cases, ranging from simple one-to-one applications such as secure storage, to complex scenarios involving time-locked disclosures, threshold access, and policy-gated content. Consider a user encrypting their medical records for personal storage: this represents the simplest case, where only the data owner should ever be able to decrypt the content. At the other end of the spectrum lie decentralized trading platforms, where encrypted orders must remain confidential until a specific onchain event to prevent frontrunning, or voting systems in which ballots remain encrypted until the voting period concludes.

Crucially, as blockchains move toward confidential transactions, decryption key management becomes a first-order usability and security challenge. In programmable private transaction systems, users must be able to reliably recover transaction secrets in order to audit, prove, or re-derive transaction state in order to spend their confidential assets. The absence of robust mechanisms for managing these decryption secrets remains one of the largest barriers to mass adoption of private transactions.

Moreover, encryption policies frequently diverge from signing policies. Threshold and multi-signature wallets are commonly used to control authorization over asset movement, yet the entities

authorized to decrypt data are not always the same as those authorized to sign transactions. In such settings, centralized systems become both fragile and inefficient, particularly when decryption must respect decentralized governance, quorum thresholds, or time-based constraints.

Finally, modern account abstraction frameworks increasingly support signing-key rotation and flexible authentication policies. This raises a fundamental question: how should encrypted data be re-encrypted, migrated, or selectively disclosed when signing keys change? Treating encryption and signing as the same concern introduces unnecessary coupling and operational risk.

All of the above scenarios illustrate a fundamental challenge: blockchain applications need encryption systems that can enforce complex, dynamic access policies while preserving the decentralized, trust-minimized properties that make blockchains valuable in the first place.

The Challenges. The absence of widely adopted encryption standards in blockchain ecosystems stems from several interconnected challenges. First, there’s uncertainty around which encryption schemes to support. Different applications have vastly different requirements: some need RSA or compatibility with existing systems, while others require ElGamal with specific elliptic-curve-based schemes for advanced cryptographic workflows. The diversity of requirements makes it difficult to define a one-size-fits-all solution.

Second, decentralized key management presents a fundamental ambiguity. Traditional approaches rely on local device storage, but this introduces challenges around device loss, backup, and cross-device access. Some projects have explored custodial services, secure enclaves, or centralized key management systems, but each approach introduces its own trade-offs in terms of usability, trust assumptions, and resilience. This challenge is further complicated by the fact that many modern authentication mechanisms (like zkLogin, Passkeys, and hardware wallets) don’t depend on persistent local storage, and their APIs often restrict the use of private keys or mnemonics to signing operations only, preventing their use for encryption purposes.

More decentralized and permissionless solutions based on secure multiparty computation (MPC) have gained attention, with projects like vetKeys or Lit Protocol, distributing trust among committees of parties. While these systems remove centralized points of failure, they typically rely on a single global committee, intertwine authentication and decryption logic, and still require developers to rely on threshold assumptions about permissionless operators.

Third, there is no common language or interface for defining access control policies over encrypted data. What does it mean for someone to have “access” to ciphertext? Should access depend on token ownership, time-based conditions, or multi-party consensus? The absence of a shared policy model forces developers to reimplement this logic for each application, creating fragmentation and limiting interoperability.

Finally, the lack of reference implementations and trusted libraries discourages adoption. Developers who want to add encryption to their blockchain applications often resort to ad hoc solutions: encrypting content with ephemeral keys and sharing those keys through side channels, or implementing bespoke key management systems that don’t generalize beyond isolated use cases.

2 Seal: A Decentralized Secrets Management Solution

Seal addresses these fundamental challenges by introducing a Decentralized Secrets Management (DSM) service that integrates Identity-Based Encryption (IBE) with threshold cryptography to enable fine-grained, secure access control over encrypted data. Unlike approaches where users encrypt a secret key under a threshold committee, IBE allows encryption directly to policy-defined identities (such as “user X after timestamp T” or “holder of NFT Y”), enabling dynamic access control without requiring users to manage separate encryption keys for each policy. At a high level, Sui smart contracts define who is allowed to decrypt, and a network of off-chain key servers enforces these rules by jointly managing decryption keys. Designed specifically for decentralized applications built on the Sui blockchain and/or Walrus decentralized storage, Seal supports use cases ranging from secure personal storage to complex policy-gated content while maintaining the trust-minimized properties that make blockchain applications valuable. Projects or receivers can define their desired key server committees per use-case, allowing multiple Seal committees to coexist and enabling flexible trust models tailored to specific application requirements.

Architecture. At its core, Seal introduces a clear architectural separation between policy definition and key management, and uses IBE to connect the two. First, let’s recall the algorithms of IBE:

- **Setup:** Generates a master secret key msk and a master public key mpk .
- **Derive(msk, id):** Given a master secret key and an identity id (string or byte array), generates a derived secret key sk for that identity.
- **Encrypt(mpk, id, m):** Given a public key, an identity and a message, returns an encryption c .
- **Decrypt(sk, c):** Given a derived secret key and a ciphertext, compute the message m . Such a scheme is correct if for any id, m and $c = \text{Encrypt}(mpk, id, m)$, we have that $\text{Decrypt}(\text{Derive}(msk, id), c) = m$.

With these IBE algorithms in place, Seal organizes encryption and decryption around the encryptor/developer, the on-chain policy, and the key servers. The key insight is that IBE identities in Seal are not simple user identifiers, but rather combinations of policy packages and policy-specific identifiers ($packageId||id$), enabling programmable access control. Seal SDKs provide encrypt and decrypt APIs for storing any secret using Seal.

When encrypting data, the encryptor/developer chooses a policy, identified by a package $packageId$, and constructs a policy-specific identifier id (e.g., an account, a timestamp, or other context). The IBE identity used for encryption is then $packageId||id$. The encryptor/developer may choose *any set* of key servers and any threshold t to encrypt its data in a way that privacy is guaranteed as long as less than t of the key servers collude, and liveness is guaranteed as long as at least t key servers are responsive. We stress again that the developer can choose any set of key servers and any threshold, depending on its preferred trust assumptions.

Access control logic is defined and enforced using programmable Move smart contracts *on-chain*, ensuring transparency, composability, and verifiability. Policies deployed under package $packageId$ control the namespace of IBE identities $packageId||*$. These policies can incorporate dynamic conditions such as timestamps, NFT ownership, voting states, or any other blockchain-accessible data.

A key server (or an MPC committee of key servers) holds the IBE master secret key msk and publishes the corresponding public key mpk . The IBE master secret key msk , and thus the ability to derive decryption keys, is never stored on-chain but resides with independent key servers operating *off-chain*. These servers may be standalone nodes operated by independent parties, or organized as decentralized permissioned or permissionless MPC committees. For example, an encryptor might select 5 servers (a mix of institutional, community-run, or MPC committees) with a threshold of 3, meaning any 3 servers can enable decryption. This flexibility allows encryptors to define their own trust assumptions rather than being forced into a one-size-fits-all model.

To decrypt, a user can request a derived key for identity $packageId||id$. When a key server receives such a request from an authenticated user, it evaluates the policy defined by package $packageId$ for identity id using the latest on-chain state of Sui and the identity of the requesting user. If the policy grants access, the key server returns the derived key to the caller; otherwise, it rejects the request. To maintain privacy, the derived key is encrypted using an ephemeral key chosen by the caller. If a threshold was required, then the user would request keys from at least t of the n key servers, and only after obtaining derived keys from at least t servers, they could decrypt the data using the Seal SDK.

While this design allows different deployments of key servers, Seal currently supports independent key servers and key servers realized as threshold MPC committees. A key server owner can configure it as permissionless (any policy may be queried) or permissioned (only allowlisted policies may be queried). An MPC committee jointly holds a single master key and serves requests under one committee public key; we detail this construction in Section 6. Future versions of Seal will add further realizations, such as enclave-based key servers and additional configurations.

Seal’s Position in the Sui Stack. Seal integrates cleanly with existing components in the Sui blockchain stack. At the blockchain layer, Seal policies are enforced via smart contracts, leveraging chain-native identities and objects powered by Sui Move. The authentication layer provides the foundation for evaluating decryption access policies using existing identity systems without introducing new authentication primitives. This is particularly valuable for authentication mechanisms like zkLogin and Passkeys, where traditional public key-based encryption may not be feasible since users may

not have published public keys before their first transaction. The storage layer, whether Sui or Walrus, handles encrypted payload storage while Seal separates key control from storage, allowing applications to maintain privacy without losing availability.

Sui Nautilus provides a framework for deploying enclaves with reproducible builds and on-chain attestation. However, enclaves face a fundamental challenge: where to securely store long-term cryptographic keys. Traditional approaches rely on external key management systems or require keys to be provisioned during enclave startup, creating security and operational complexities. Seal solves this by allowing enclaves to derive their long-term keys through policy-based access control. An enclave can request a Seal key using its attested identity (PCRs and public key), enabling use cases like encrypted data lakes that can only be processed by specific, verified enclave binaries. The flexibility of Seal policies allows the use of permissioned or permissionless enclaves.

Conversely, Nautilus can offer additional key server deployments by enabling enclave-based key servers. For example, a Seal key server running inside a Nautilus enclave can hold its master key securely while also being backed up by multiple other Seal key servers. This creates a hybrid trust model where users get the security benefits of distributed trust across multiple key servers, combined with the operational efficiency of a single enclave-based server. The enclave’s attested identity ensures that only the correct binary can access the master key, while Seal’s threshold cryptography provides resilience against enclave compromise or failure. Other combinations of Nautilus and Seal include use cases like poker shuffling, where enclaves can securely manage game state while using Seal for key distribution. Looking forward, Seal’s identity-based approach makes it uniquely suited for confidential transaction key management, especially in crypto-agile systems where users may employ multi-sig, passkeys, or zkLogin. In these scenarios, a user’s public key may not be known before their first transaction, making traditional public key encryption infeasible. Seal enables such users to receive blinding factors and other cryptographic materials through policy-based access control, addressing a key challenge for on-chain privacy in systems that support diverse authentication mechanisms.

3 Example Policies

This section showcases how Seal leverages the Move language to define expressive and enforceable access control policies. By encoding logic directly into on-chain programs, Seal enables decentralized and auditable rules for decrypting data. Policies can share code and on-chain data from other contracts, using the same toolchain.

To define an access policy in Seal, developers implement a function with the prefix `seal_approve` within a Move package. This function acts as a gatekeeper for key access: it is called by the key server to determine whether a request should be approved. The `seal_approve` function performs the logic evaluation and receives a key identifier `id`, which corresponds to the policy-specific identifier in the IBE identity `packageId||id` (where `packageId` is the package identifier and `id` is the policy-specific identifier passed to the function), blockchain state (e.g., the current time from the `Clock` object), etc. If the result is false, the function aborts the transaction with an error code.

Policies are namespaced by packages, segregating policies defined by different contracts. Each policy defines how to construct the IBE key identifier (the `id` used for encryption and decryption). This identifier is derived from contextual data encoded in Move and typically follows a structured format, for example, a `timestamp` for time-lock encryption. The format is intentionally modular, enabling policy- or app-specific semantics for key identifiers.

In the following we present example policies. Additional use cases include event-released encryption, where decryption becomes available when specific on-chain conditions are met (e.g., when more than 100 users enroll in a program, when a game character reaches a particular level or position on the map, when liquidity falls below a threshold such as 10M, when a DAO proposal passes with sufficient votes, or when a crowdfunding campaign reaches its funding goal).

Account Based Access. This policy ensures that only the owner of account x can access the derived key for identity x . This is useful for one-to-one applications like personal storage, private messaging, or private document sharing. For example, anyone can encrypt a message that only Bob can decrypt by using Bob’s account address as the IBE identity. The policy can be implemented as follows:

```
entry fun seal_approve(id: vector<u8>, ctx: &TxContext) {
```

```

    let caller_bytes = bcs::to_bytes(&ctx.sender());
    assert!(id == caller_bytes, ENoAccess);
}

```

The *id* argument is the IBE identity requested under the package’s namespace and is the only mandatory argument. Additional Move arguments can be used by policies as needed, for example the transaction context.

NFT Gated Access. This policy ensures that only owners of a specific single-owner object type can access the keys governed by a package. This is useful, for example, for NFT-based paywalls. The policy can be implemented as follows:

```

entry fun seal_approve(id: vector<u8>, nft: &NFT) {
    // Ownership of nft is checked by the Move VM.
    // Nothing else needs to be checked here.
}

```

More sophisticated policies can incorporate fields from the NFT object (e.g., creation time for subscription renewal windows).

Time-lock Encryption. This policy allows users to encrypt data to a future point in time. Anyone can decrypt the data once the specified time has passed. This is useful, for example, for secure voting or MEV-aware workflows where data must remain private until an on-chain condition is met. The policy can be implemented as follows:

```

entry fun seal_approve(id: vector<u8>, c: &clock::Clock): bool {
    let mut prepared: BCS = bcs::new(id);
    let t = prepared.peel_u64();
    let leftovers = prepared.into_remainder_bytes();
    (leftovers.length() == 0) && (c.timestamp_ms() >= t)
}

```

Users encode the future time as the identity they use for encryption. The above policy interprets *id* as a *u64* timestamp and compares it with Sui’s on-chain clock. If the specified time has passed, access is granted. Similar policies can use Sui objects created after a particular on-chain event (e.g., when a threshold number of encrypted votes has been cast).

4 Key Server and SDKs

Stateless key server by design. As stated in Section 2, key servers only store the master secret key *msk* and do not need to keep state across requests or sessions. Instead, they derive decryption keys on demand from *msk*.

To determine whether a key request satisfies the relevant policy, the key server evaluates (off-chain, with the help of a Sui full node) the associated `seal_approve` Move function on the Sui blockchain using the latest state of the chain. The result determines whether the key can be released, ensuring that policy checks are enforced without any state changes or gas usage on-chain, since the evaluation is performed as a read-only dry run.

This approach allows for simple horizontal scalability and robustness: each key server operates independently and can verify whether a request meets the access policy without requiring shared state or user-specific metadata.

Key server as an MPC committee. A key server can also be operated as a threshold MPC committee, in which a set of *n* parties jointly holds the master secret key and any *t* of them can serve a request. The committee runs a distributed key generation and supports key rotation, so members can change without altering the master key (Section 6). Clients are oblivious to this: they encrypt to, and request keys from, a single committee public key exactly as for a standalone server, so the SDK flow and the policy evaluation above are unchanged.

How data is encrypted. To encrypt data in Seal, the encryptor uses Seal SDKs to construct a policy-specific key identifier *id* (which, together with the package identifier *packageId*, forms the IBE identity *packageId||id* as described in Section 2) that encodes the access conditions (e.g., a timestamp for time-lock using `bcs::to_bytes(T)`). The encryptor then selects a set of key servers and a threshold value, specifying how many servers must cooperate for decryption. Using Seal SDKs, the encryptor performs a KEM/DEM encryption (detailed in Section 5): (i) generates a random symmetric key *k_sym* and encrypts the data with it using AES-256-GCM or HMAC-256-CTR, (ii) uses the TSS-BF-KEM scheme to encrypt *k_sym* under the chosen *packageId||id*, creating separate ciphertexts for each selected key server using Boneh-Franklin IBE with BLS12-381 curves, and (iii) packages everything into a single object containing the policy identifier, threshold parameters, and the encrypted data. The resulting ciphertext can be stored anywhere since the decryption keys are held off-chain by the key servers. This design ensures that the encryptor never needs to communicate with key servers during encryption: the policy logic and key derivation happen entirely during the decryption phase.

How data is decrypted. Clients submit a policy invocation bundle to key servers consisting of a minimal Programmable Transaction Block (*PTB*, `txBytes`) that calls the policy entry function (e.g., `<package>::module::seal_approve`) with the policy’s inputs, and any required references to on-chain objects used by the policy (e.g., `Clock`, `NFT`). Evaluation proceeds as follows:

1. If the policy requires the caller identity, validate an attestation bound to the request (e.g., a chain-native signature); otherwise may skip.
2. Decode `txBytes` and ensure it contains only a `MoveCall` command to a function with the prefix `seal_approve`.
3. Fetch the latest on-chain state for any objects referenced in the PTB and perform a read-only dry run using `dry_run_transaction_block`. Success is defined as “no abort” of the function; a policy failure manifests as an abort.
4. If all checks pass, the server derives and returns its IBE key share for *packageId||id*; otherwise the request is rejected. (To maintain consistency of identities across package upgrades, the value of *packageId* used for key derivation is always set to the value of the first published version of the package.)

5 Cryptography

Seal integrates Encrypted BLS signatures and Threshold Secret-shared Identity-Based Encryption (IBE) to offer a practical solution balancing rigorous security properties with operational efficiency. We define the cryptographic primitives explicitly, including key generation, encryption, signing, and verification algorithms, and provide formal proofs of their correctness and security guarantees.

Specifically, we present an ElGamal-based Encrypted BLS signature scheme that ensures unforgeability and privacy, relying on standard cryptographic assumptions in the Random Oracle Model. Additionally, our Threshold Secret-shared IBE scheme combines Boneh-Franklin IBE, specifically the KEM-DEM hybrid encryption adaptation due to [LQ05], with threshold secret sharing to achieve strong security in the Universally Composable framework [Can01]. Our formal proofs demonstrate that Seal meets these robust security definitions, establishing a foundation for secure blockchain protocols requiring both high performance and stringent security standards.

TSS-BF-KEM. The Threshold Secret-Shared Boneh-Franklin Key Encapsulation Mechanism (TSS-BF-KEM) is the core cryptographic primitive that enables Seal’s threshold decryption capabilities. It combines three key components:

- **Boneh-Franklin IBE:** Provides identity-based encryption where any string can serve as a public key, eliminating the need for public key infrastructure. In Seal, the policy-specific identifier *id* serves as the IBE identity.
- **Threshold Secret Sharing:** Uses Shamir’s secret sharing to split the symmetric encryption key *k_sym* into shares, such that any *t* out of *n* shares can reconstruct the original key, but fewer than *t* shares reveal no information about the key.

- **Key Encapsulation Mechanism (KEM):** Separates the encryption of the symmetric key from the encryption of the actual data, providing better efficiency and enabling the threshold sharing of the key material.

The TSS-BF-KEM process works as follows: The encryptor/developer first samples a symmetric key k_{sym} , then splits it into n shares using Shamir’s secret sharing with threshold t . Each share is then encrypted using Boneh-Franklin IBE under the policy identity id , but using the master public key of a different key server. This creates n separate ciphertexts, one for each key server. During decryption, a user must obtain at least t of these ciphertexts and the corresponding IBE private keys from the key servers to reconstruct k_{sym} and decrypt the original data.

This design provides several key advantages: (1) **Key Independence:** Each key server maintains its own IBE master key, eliminating the need for complex distributed key generation protocols; (2) **Flexible Trust Models:** Users can choose their own set of key servers and threshold values based on their trust assumptions; (3) **Privacy:** The actual data remains encrypted with the symmetric key, and only the key shares are distributed across servers; (4) **Efficiency:** The KEM/DEM structure allows for efficient encryption of large data while keeping the threshold operations on small key material.

Encrypted BLS Signatures. While TSS-BF-KEM handles the encryption and threshold decryption of data, Encrypted BLS signatures provide the secure mechanism for key derivation in Seal’s architecture. The extracted keys corresponding to identities can be viewed as partial BLS signatures on the requested identity.

The security challenge is ensuring that no eavesdropper can learn the partial signatures and aggregate them without proper authentication. To address this, Seal employs an encryption scheme where the requester sends a signed augmented ElGamal public key in their request. This augmentation enables an untrusted verifier to verify the encrypted partial signatures and aggregate them (in the case of MPC scenarios) without decrypting them, while ensuring that only the authenticated requester can decrypt the partial signatures and construct the full signature.

This design ensures that key servers can securely distribute IBE private keys while maintaining both privacy and verifiability properties.

Future Work. Building on the foundation established by our current TSS-BF-KEM, Encrypted BLS, and MPC committee implementations, we plan to extend Seal’s cryptographic primitives to support additional encryption schemes and security models. This will allow users to choose their own trust assumptions and security requirements based on their specific use cases, while maintaining Seal’s core architectural principles.

A key area of future development is **Post-Quantum Security:** post-quantum secure TSS-IBE schemes to ensure long-term security against quantum attacks.

5.1 Encrypted BLS Service

In the Seal system, there are key servers which act as authorities for IBE keys. Clients obtain sub-keys which are extracted corresponding to queried ID’s. We use an adaptation of the Boneh-Franklin IBE [BF01], for which the extracted sub-key for an ID is essentially a BLS [BLS01] signature on the ID. An untrusted aggregator service may help the client obtain sub-keys from several key servers. Crucially, we don’t want the aggregators to have access to these sub-keys. In order to hide the sub-keys in transport from key servers to clients, the clients generate ephemeral encryption keys and send them as part of their request. The key servers hide the sub-key responses with the ephemeral key.

Definition 1 (Encrypted BLS). *An Encrypted_BLS scheme is a tuple of algorithms (keygen , ephkey , encsign , verify , decrypt):*

- $\text{keygen}(\lambda) \rightarrow (pk, sk)$: takes the security parameter λ as input and outputs BLS keys (pk, sk) .
- $\text{ephkey}(\lambda) \rightarrow (ephpk, ephsk)$: outputs ephemeral keys $(ephsk, ephpk)$.
- $\text{encsign}(sk, ephpk, ID) \rightarrow \text{encsig}$: takes a secret key sk , an ephemeral public key $ephpk$ and an ID and outputs an encrypted signature encsig on ID.

- $\text{encverify}(pk, \text{ephp}, ID, \text{encsig}) \rightarrow \{0, 1\}$: takes $(pk, \text{ephp}, ID, \text{encsig})$, and checks whether the encrypted BLS signature verifies.
- $\text{decrypt}(\text{ephsk}, \text{encsig}) \rightarrow \text{sig}$: takes $(\text{ephsk}, \text{encsig})$, and outputs a BLS signature sig .
- $\text{verify}(pk, ID, \text{sig}) \rightarrow \{0, 1\}$: takes (pk, ID, sig) , and checks whether the BLS signature verifies.

The ephemeral encryptions have a design that enables aggregators to verify them without accessing the underlying signatures. The encryption is useful even when no aggregator is used. In particular, the encryption protects against a malicious key server or an eavesdropper that does a replay attack and sends the same user request to other key servers.

Definition 2 (Correctness). *If keygen , ephkey , encsign , and decrypt operations of an Encrypted_BLS scheme follow protocol, then encverify and verify both return 1.*

Definition 3 (EncVerify Soundness). *An adversary has negligible advantage in winning the following game against a challenger:*

1. Challenger samples $(\text{ephp}, \text{ephsk}) \leftarrow \text{ephkey}(\lambda)$ and sends ephp to Adversary.
2. Adversary outputs (pk, ID, encsig) .
3. Challenger computes $\text{sig} \leftarrow \text{decrypt}(\text{ephsk}, \text{encsig})$
4. Adversary wins if $\text{verify}(pk, ID, \text{sig})$ doesn't hold but $\text{encverify}(pk, \text{ephp}, ID, \text{encsig})$ holds.

This definition ensures that a valid encsig (i.e. one that passes the encverify check) will necessarily contain the encryption of a valid signature sig (i.e. the encrypted sig passes the verify check). This implies that service is able to verify the consistency of a key server's response without getting access to the final signature.

The main security properties of Encrypted BLS, including unforgeability and privacy, are formalized later as part of the overall ideal functionality of the system in Section 5.3.

5.1.1 ElGamal-based Construction

We now describe an Encrypted BLS service based on ElGamal encryption. Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order q , let g_1, g_2 be generators of $\mathbb{G}_1, \mathbb{G}_2$ and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be an efficiently computable bilinear map. Let $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ be a hash-to-curve function. This construction enables aggregators to homomorphically aggregate encrypted signatures, an operation that enables MPC committees for secret shared BLS keys with a single committee public key.

$\text{keygen}(\lambda)$: sample $sk \leftarrow \mathbb{Z}_q$ and set $pk \leftarrow g_2^{sk}$.

$\text{ephkey}(\lambda)$: Sample $x \leftarrow \mathbb{Z}_q$ and set $\text{ephsk} \leftarrow x$ and $\text{ephp} \leftarrow (g_1^x, g_2^x)$.

$\text{encsign}(sk, \text{ephp}, ID)$: Sample $r \leftarrow \mathbb{Z}_q$. Parse ephp as $(\text{ephp}_1, \text{ephp}_2)$. Output $\text{encsig} \leftarrow (g_1^r, \text{ephp}_1^r \cdot H(ID)^{sk})$.

$\text{encverify}(pk, \text{ephp}, ID, \text{encsig})$: Check if $e(\text{encsig}_2, g_2) = e(H(ID), pk) \cdot e(\text{encsig}_1, \text{ephp}_2)$.

$\text{decrypt}(\text{ephsk}, \text{encsig})$: Output $\text{sig} \leftarrow \text{encsig}_2 / (\text{encsig}_1)^{\text{ephsk}}$.

$\text{verify}(pk, ID, \text{sig})$: Check if $e(\text{sig}, g_2) = e(H(ID), pk)$.

It is easy to see that the construction is correct. We will prove it's main security goal in Section 5.3, but state and prove an additional security guarantee here:

Theorem 1 (EncVerify Soundness). *The ElGamal-based Encrypted BLS scheme satisfies EncVerify soundness.*

Proof. Assume that encverify holds with $\text{ephkey} = (g_1^x, g_2^x)$ supplied by Challenger and (pk, ID, encsig) supplied by the Adversary. This means:

$$e(\text{encsig}_2, g_2) = e(H(ID), pk) \cdot e(\text{encsig}_1, g_2^x) = e(H(ID), pk) \cdot e((\text{encsig}_1)^x, g_2)$$

Therefore,

$$e(\text{encsig}_2 / (\text{encsig}_1)^x, g_2) = e(H(ID), pk) \implies e(\text{sig}, g_2) = e(H(ID), pk)$$

Thus verify is also satisfied. $\square$

5.2 Threshold Secret-shared IBE

Definition 4 (Threshold Secret-shared IBE). A TSS-IBE scheme is a tuple of algorithms (*keygen*, *extract*, *admissible*, *encrypt*, *decrypt*):

keygen(λ) $\rightarrow (sk, pk)$: Generate IBE keypairs for a server, given a security parameter.

extract(sk, ID) $\rightarrow sk_{ID}$: Generate an IBE key for ID corresponding to server sk .

admissible($\vec{sk}_{ID}, \vec{pk}, ID, t$) $\rightarrow \{0, 1\}$: Let n be the dimension of the vector $\vec{pk}$, which is a vector of public keys. Return 1 if $\vec{sk}_{ID}$ is also of dimension n and there are at least t indices $i \in [n]$, such that $(\vec{sk}_{ID})_i$ is an extracted secret key for ID corresponding to $(\vec{pk})_i$ and the rest $n - t$ of them equal to $\perp$.

encrypt($\vec{pk}, t, ID, m, aad$) $\rightarrow c = (\vec{pk}, t, c')$: Return the encryption of a message m with ID characterizing the target recipient. We assume that c includes the ciphertext c' , the vector of public keys used to construct it, $\vec{pk}$ and the intended threshold t . The aad field is an optional way to uniquely label ciphertexts. This is helpful in situations where we want to prevent replay of encryptions, such as encrypted votes.

decrypt($\vec{sk}_{ID}, ID, c, aad$) $\rightarrow m$: Decrypt a ciphertext $c.c'$ given an admissible subset of the user secret keys for the corresponding key servers, that is, *admissible*($\vec{sk}_{ID}, c.\vec{pk}, ID, c.t$) = 1.

Definition 5 (Correctness of TSS-IBE). Correctness holds if for all $\lambda, n, t, \vec{pk}$ with $|\vec{pk}| = n, ID, \vec{sk}_{ID}$, the following holds: if

$$\begin{aligned} (sk_i, pk_i) &\leftarrow \text{keygen}(\lambda), \text{ for all } i \in [n] \\ c &\leftarrow \text{encrypt}(\vec{pk}, t, ID, m, aad), \text{ such that } (\vec{pk})_i \in pk_{[n]} \text{ for all } i \in [n] \\ sk_{ID,i} &\leftarrow \text{extract}(sk_i, ID), \text{ for all } i \in [n] \\ 1 &\leftarrow \text{admissible}(\vec{sk}_{ID}, \vec{pk}, ID, t) \\ m' &\leftarrow \text{decrypt}(\vec{sk}_{ID}, ID, c, aad) \end{aligned}$$

then we must have $m = m'$.

5.2.1 Construction TSS-BF-KEM-CCA

In this section, we build a TSS-IBE based on Boneh-Franklin IBE, a threshold secret sharing scheme TSS over $\mathbb{F}_{2^\lambda}$, a symmetric labeled encryption scheme SYMENC with message and ciphertext domains $\{0, 1\}^*$, and Random Oracles $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1, H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda, H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$.

KeyGen(λ): Given bilinear groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g_1, g_2, e)$,

- Sample $sk \leftarrow \mathbb{Z}_q^n$.
- Compute $pk \leftarrow g_2^{sk}$.
- Output (pk, sk) .

Extract(i, sk_i, ID): Output $H_1(ID)^{sk_i}$.

Encrypt($\vec{pk}, t, ID, m, aad$): Given a vector of public keys $\vec{pk}$, which might repeat some pk 's to represent weight, a threshold t , an ID , a plaintext m , and an additional authenticated data field aad ,

- Sample $k \leftarrow \{0, 1\}^\lambda, r \leftarrow \mathbb{Z}_q$
- Sample $(k_1, \dots, k_n) \leftarrow \text{TSS.share}(k, n, t)$
- Compute $\text{nonce} \leftarrow g_2^r$
- Compute $c'_i = k_i \oplus H_2(i, (\vec{pk})_i, H_1(ID), \text{nonce}, e(H_1(ID), (\vec{pk})_i^r))$ for all $i \in [n]$
- $(k_r, k_{sym}) \leftarrow H_3(k, \vec{pk}, t, c'_{[n]})$.
- $c_r \leftarrow r \oplus k_r$

- $sem = \text{SYMENC.encrypt}(k_{sym}, m, aad)$ where aad is the encryption label.
- Output $(\vec{pk}, t, nonce, c_r, c'_{[n]}, sem)$

Admissible $(\vec{sk}_{ID}, \vec{pk}, ID, t)$: Given a set of secret keys extracted for ID ,

- Set $count = 0$
- For all $i \in [|\vec{pk}|]$, do:
 - if $((\vec{sk}_{ID})_i \neq \perp)$, then:
 - * if $e((\vec{sk}_{ID})_i, g_2) \neq e(H_1(ID), (\vec{pk})_i)$, then return 0.
 - * else set $count \leftarrow count + 1$
- If $count \neq t$, then return 0, else return 1

Decrypt $(\vec{sk}_{ID}, ID, (\vec{pk}, t, nonce, c_r, c'_{[n]}, sem), aad)$: Given a set of t secret keys extracted for ID , a ciphertext, and an additional authenticated data field aad ,

- If **admissible** $(\vec{sk}_{ID}, \vec{pk}, ID, t) = 0$, then return $\perp$.
- Compute $k_i \leftarrow c'_i \oplus H_2(i, (\vec{pk})_i, H_1(ID), nonce, e(sk_{ID,i}, nonce))$, for all $(\vec{sk}_{ID})_i \neq \perp$
- Compute $k \leftarrow \text{TSS.reconstruct}(\{k_i\}_{(\vec{sk}_{ID})_i \neq \perp}, n, t)$
- $(k_r, k_{sym}) \leftarrow H_3(k, \vec{pk}, t, c'_{[n]})$.
- Compute $r \leftarrow c_r \oplus k_r$
- Assert $nonce = g_2^r$
- Compute $k_{[n]} \leftarrow \text{TSS.extend}(\{k_j\}_{(\vec{sk}_{ID})_j \neq \perp})$
- Assert $k_i = c'_i \oplus H_2(i, (\vec{pk})_i, H_1(ID), nonce, e(H_1(ID), (\vec{pk})_i^r))$, for all $(\vec{sk}_{ID})_i = \perp$
- Output $\text{SYMENC.decrypt}(k_{sym}, sem, aad)$

In particular, the SYMENC is implemented in Seal with two options: HMAC-CTR and AES-GCM as follows:

Algorithm 1 AuthenticatedEncrypt

```

1: Input:  $M \in \{0, 1\}^*$ ,  $aad \in \{0, 1\}^*$ ,  $k \in \{0, 1\}^\lambda$ ,  $mode \in \{\text{AES}, \text{HMAC}\}$ 
2: Output:  $sem$ 
3: if  $mode = \text{AES}$  then
4:    $sem \leftarrow \text{AES\_256\_GCM.encrypt}(k, m, aad)$  ▷ AES-GCM encryption
5: else
6:   Parse  $m_{[l]} \leftarrow M$ 
7:   for  $i \in [l]$  do
8:      $c_i \leftarrow m_i \oplus \text{HMAC\_SHA3\_256}(k, \text{"enc"} \mid i)$ 
9:   end for ▷ Stream cipher encryption
10:   $mac \leftarrow \text{HMAC\_SHA3\_256}(k, \text{"mac"} \mid aad \mid c_{[l]})$  ▷ Compute MAC
11:   $sem \leftarrow (c_{[l]}, mac)$ 
12: end if
13: return  $sem$ 

```

Algorithm 2 AuthenticatedDecrypt

```
1: Input:  $sem, aad \in \{0, 1\}^*, k \in \{0, 1\}^\lambda, mode \in \{AES, HMAC\}$ 
2: Output:  $M \in \{0, 1\}^*$  or  $\perp$ 
3: if  $mode = AES$  then
4:    $M \leftarrow AES\_256\_GCM.decrypt(k, sem, aad)$  ▷ AES-GCM decryption
5:   if  $M = \perp$  then
6:     return  $\perp$  ▷ Authentication failed
7:   end if
8: else
9:   Parse  $(c_{[l]}, mac) \leftarrow sem$ 
10:   $mac' \leftarrow HMAC\_SHA3\_256(k, "mac" \parallel aad \parallel c)$  ▷ Recompute MAC
11:  if  $mac' \neq mac$  then
12:    return  $\perp$  ▷ Authentication failed
13:  end if
14:  for  $i \in [l]$  do
15:     $m_i \leftarrow c_i \oplus HMAC\_SHA3\_256(k, "enc" \parallel i)$ 
16:  end for ▷ Stream cipher decryption
17:   $M \leftarrow m_{[l]}$ 
18: end if
19: return  $M$ 
```

Theorem 2. *TSS-BF-KEM-CCA satisfies correctness.*

Proof. The correctness of IBE decryption follows immediately from Boneh-Franklin. Decrypting with an admissible set of extracted secret keys allows computation of t shares of n Shamir secret shares of k . Thus k can be correctly reconstructed.

Since *nonce* is a binding commitment to r and the share consistency checks mirror the encryption process, these checks will also pass.

Now given the decryption correctness of SYMENC and the correct reconstruction of k , we get the correct decryption to the plaintext m . $\square$

5.3 Ideal Functionality and Protocol

This section presents the ideal functionality and protocol for verifiable threshold secret sharing identity-based encryption (vetssIBE), which enables secure key derivation and encryption in a distributed setting. The construction allows multiple servers to jointly derive encryption keys for specific identities while maintaining security guarantees. The protocol Π_{tssIBE} is an integrated standalone protocol that directly implements the vetssIBE functionality. We present the ideal functionality specification, the concrete protocol implementation, and a simulator that demonstrates UC-security under the co-BDH assumption in the Random Oracle Model.

The ideal functionality $\mathcal{F}_{\text{tssIBE}}$ provides verifiable threshold secret sharing identity-based encryption (vetssIBE) in the UC framework. It enables multiple servers to individually derive encryption keys for specific identities while maintaining security guarantees. The functionality supports three main operations: (1) **Key Derivation:** Servers can derive encryption keys for specific identities on behalf of users, (2) **Encryption:** Any party can encrypt messages for specific identities using given public keys, and (3) **Decryption:** Users can decrypt messages if they have derived the necessary keys from the required servers. The design ensures that only users with proper key derivations can decrypt messages, while maintaining verifiability and security properties.

The functionality maintains several key data structures to track the state of the system:

- **Honest MPKs:** MPK stores the public keys of the honest servers.
- **Key Derivation Tracking:** EKD tracks which identities are currently being derived for each user-server pair, enabling proper simulation in case of corrupt user requesting for a key that was used in a simulated encryption.

- **Ciphertext Storage:** *CTXT* stores all encrypted messages with their associated metadata (public keys, threshold, ciphertext, identity, message, AAD) for proper decryption and consistency.

Intuition behind the functionality. The ideal functionality $\mathcal{F}_{\text{tssIBE}}$ formalizes a core security property of the system that ensures confidentiality in a distributed threshold setting. At its heart, the functionality guarantees that for any identity for which no corrupt party has obtained an extracted key, the adversary cannot learn the underlying message, even when observing encrypted ciphertexts. This security property is fundamental to the system's design and forms the basis for secure key distribution in multi-server environments.

The security guarantee is enforced via a TSS-BF-KEM (Threshold Secret Sharing Boneh-Franklin Key Encapsulation Mechanism) scheme. This construction operates in a t -out-of- n threshold setting, where decryption requires possession of at least t out of n extracted keys from different servers. The threshold structure ensures that if corrupt users have not obtained the required minimum number of extracted keys, then ciphertexts encrypted to the full set of n servers cannot be decrypted, even by a coalition of corrupt parties. This provides security against partial corruption scenarios where some but not all servers may be compromised. Our Boneh-Franklin IBE modification provides non-malleability, ensuring that adversaries cannot transform valid ciphertexts into other valid ciphertexts for related messages or identities. This property is captured by the ideal functionality's requirement that decryption only succeeds for ciphertexts that were legitimately created and stored in *CTXT*. The functionality maintains state tracking through several interconnected data structures, as explained above. A central ciphertext table stores all encrypted message pairs along with their associated metadata, including the public keys used for encryption, the target identity, the original message, and any additional authenticated data. This table serves as the record for determining legitimate decryption operations and prevents inconsistencies that could arise.

A particularly challenging scenario in the security proof arises when a corrupt user requests a key that was previously used in a simulated encryption operation. In this case, the functionality must reveal the decryption to the simulator to maintain indistinguishability, but this creates a complex technical challenge: the simulator must be able to equivocate previously simulated ciphertexts to open correctly to the newly supplied messages. This equivocation process requires programming of the random oracles and careful management of the symmetric encryption components to ensure that the previously generated ciphertexts can be made to decrypt to the correct messages without the adversary detecting any inconsistencies in the simulation. This functionality is an integration of the vetKeys [CCN⁺23] $\mathcal{F}_{\text{bls}}$ and $\mathcal{F}_{\text{vetibe}}$ functionalities and adapted to independent signers and our TSS-BF-KEM scheme, rather than MPC-based signers. The formal specification of the ideal functionality $\mathcal{F}_{\text{tssIBE}}$ is given in Figure 1.

Protocol Construction. The protocol Π_{tssIBE} is constructed by composing two cryptographic primitives: *Encrypted BLS* (described in Section 5.1.1) and *TSS-BF-KEM* (Threshold Secret Sharing Boneh-Franklin Key Encapsulation Mechanism, described in Section 5.2.1). While these components serve distinct purposes, they are integrated into a unified protocol rather than being composed as separate hybrid functionalities. This unified design simplifies the security analysis by avoiding the complexity of composing multiple ideal functionalities, and it allows the extracted keys from Encrypted BLS to be directly used as IBE secret keys in TSS-BF-KEM without additional transformation layers.

The *Encrypted BLS* primitive is used in the "ekderive" operation to securely deliver IBE extracted keys from servers to users. Specifically, when a user $\mathcal{U}$ requests an extracted key for identity id from server $\mathcal{S}_i$, the protocol follows the ElGamal-based Encrypted BLS construction: (1) $\mathcal{U}$ generates an ephemeral transport key pair $tpk = (g_1^{tsk}, g_2^{tsk})$ where tsk is sampled from $\mathbb{Z}_q$, (2) $\mathcal{S}_i$ computes the IBE extracted key as $\sigma = H_1(id)^{sk_i}$ (which is essentially a BLS signature on id), (3) $\mathcal{S}_i$ encrypts this key using ElGamal encryption to produce $es = (g_1^r, tpk_1^r \cdot \sigma)$ where r is sampled from $\mathbb{Z}_q$, and (4) $\mathcal{U}$ decrypts to recover $K_{i,id} = e_2/e_1^{tsk} = \sigma$. This encrypted transport mechanism ensures that the extracted keys remain confidential during transmission, protecting against eavesdroppers and malicious aggregators, while enabling verifiability through the **encverify** check.

The *TSS-BF-KEM* primitive handles the actual encryption and threshold decryption of messages. The "encrypt" operation directly invokes `TSSBFKEM.encrypt( $mpk, t, id, m, aad$ )` to produce a ciphertext C that encapsulates a symmetric key k using a threshold secret sharing scheme, then encrypts

Ideal Functionality $\mathcal{F}_{\text{tssIBE}}$

- On $(sid, \text{"init"}, \mathcal{S}_i)$ from honest $\mathcal{S}_i$:
 Send $(sid, \text{"init"}, \mathcal{S}_i)$ to Sim. If mpk_i isn't defined, wait for a response mpk_i from Sim and store mpk_i . If MPK is not initialized, initialize $EKD, MPK, CTXT \leftarrow \emptyset$. Add mpk_i to MPK . Output $(sid, \text{"output-mpk"}, mpk_i)$ to $\mathcal{S}_i$.
- On $(sid, \text{"ekderive"}, \mathcal{S}_i, id, \mathcal{U})$ from honest $\mathcal{S}_i$:
 Add $(mpk_i, id, \mathcal{U})$ to EKD and send $(sid, \text{"ekderive"}, \mathcal{S}_i, id, \mathcal{U})$ to Sim.
 Output $(sid, \text{"ekderive"}, \mathcal{S}_i, id)$ to $\mathcal{U}$.
- On $(sid, \text{"encrypt"}, \vec{mpk}, t, id, m, aad)$ from $\mathcal{P}$:
 Let $I = \{i \in [|\vec{mpk}|] : (mpk)_i \notin MPK \text{ OR } \exists \text{ corrupt } \mathcal{U}' \text{ with } (mpk_i, id, \mathcal{U}') \in EKD\}$.
 If $|I| \geq t$, send $(sid, \text{"encrypt"}, \vec{mpk}, t, id, m, aad)$ to Sim.
 Otherwise send $(sid, \text{"encrypt"}, \vec{mpk}, t, id, |m|, aad)$ to Sim.
 Wait for response C from Sim such that $\nexists (mpk, t, C, id, \cdot, aad) \in CTXT$. Add $(\vec{mpk}, t, C, id, m, aad)$ to $CTXT$ and output $(sid, \text{"encrypt"}, \vec{mpk}', id, m, aad, C)$ to $\mathcal{P}$.
- On $(sid, \text{"decrypt"}, \vec{\mathcal{S}}, \vec{mpk}, t, id, C, aad)$ from $\mathcal{U}$:
 Let $S_{\mathcal{U}} = \{\mathcal{S}_i \in \vec{\mathcal{S}} : (mpk_i, id, \mathcal{U}) \in EKD\}$ and $S_{\text{corrupt}} = \{\mathcal{S}_i \in \vec{\mathcal{S}} : \exists \text{ corrupt } \mathcal{U}' \text{ with } (mpk_i, id, \mathcal{U}') \in EKD \text{ OR } mpk_i \notin MPK\}$.
 If $\mathcal{U}$ is honest and $|S_{\mathcal{U}}| < t$, then ignore.
 Else if $\mathcal{U}$ is corrupt and $|S_{\text{corrupt}}| < t$, then ignore.
 Else if $\exists (mpk, t, C, id, m, aad) \in CTXT$, then output $(sid, \text{"decrypt"}, C, id, m, aad)$ to $\mathcal{U}$.
 Else, send $(sid, \text{"decrypt"}, \vec{\mathcal{S}}, \vec{mpk}, t, C, id, aad)$ to Sim and await a response m . Add $(\vec{mpk}, t, C, id, m, aad)$ to $CTXT$ and output $(sid, \text{"decrypt"}, C, id, m, aad)$ to $\mathcal{U}$.

Figure 1: The ideal functionality $\mathcal{F}_{\text{tssIBE}}$ for verifiable threshold secret sharing identity-based encryption (vetssIBE) in the UC framework. The functionality supports key derivation, encryption, and decryption operations while maintaining security guarantees through state tracking in MPK , EKD , and $CTXT$.

the message m under that key. The "decrypt" operation uses the extracted keys $\vec{K}_{id} = \{K_{i,id}\}_{i \in [n]}$ (obtained via the Encrypted BLS mechanism) to invoke $\text{TSSBFKEM.decrypt}(\vec{K}_{id}, id, C, aad)$, which reconstructs the symmetric key from at least t shares and decrypts the message.

The integration of these primitives is direct: the extracted keys $K_{i,id}$ obtained through Encrypted BLS are exactly the IBE secret keys $sk_{ID,i} = H_1(ID)^{sk_i}$ required by TSS-BF-KEM for decryption. This design ensures that users who have successfully derived keys from at least t servers can decrypt ciphertexts, while maintaining security guarantees even when some servers or aggregators are corrupt. The protocol Π_{tssIBE} is therefore an *integrated standalone protocol* that directly implements the vetssIBE functionality, rather than being a hybrid composition of separate ideal functionalities. The complete protocol specification is given in Figure 2.

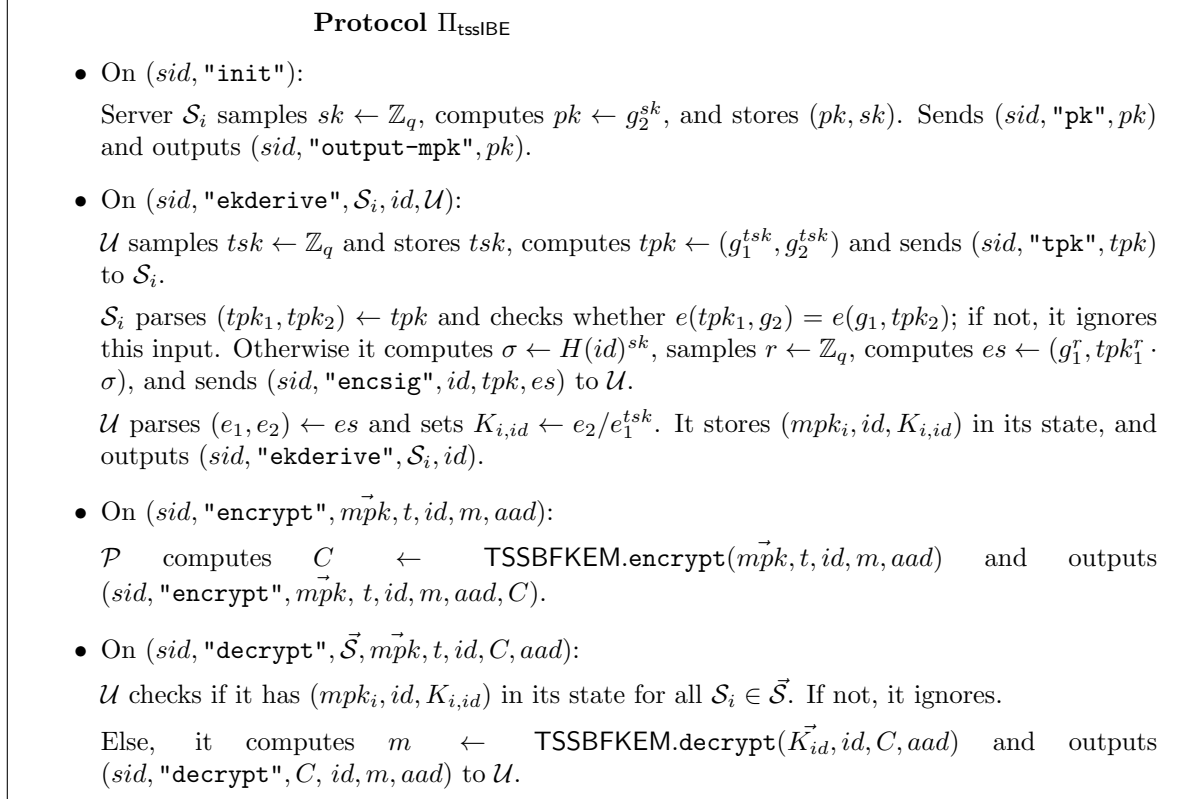


Figure 2: The protocol Π_{tssIBE} implementing verifiable threshold secret sharing identity-based encryption (vetssIBE). The protocol integrates Encrypted BLS for secure key derivation and TSS-BF-KEM for threshold encryption and decryption.

5.4 Security Proof

We now establish the security of the protocol Π_{tssIBE} by proving that it UC-realizes the ideal functionality $\mathcal{F}_{\text{tssIBE}}$ under standard cryptographic assumptions. The proof proceeds in two main steps: first, we construct a simulator Sim that interacts with the ideal functionality and the environment, demonstrating that any attack in the real world can be simulated in the ideal world. Second, we show that any distinguisher between the real and ideal executions can be used to solve the co-BDH (co-Bilinear Diffie-Hellman) problem, which is the same assumption used in Boneh-Franklin IBE.

Theorem 3 (UC-Security of vetssIBE). *The protocol Π_{tssIBE} UC-realizes the ideal functionality $\mathcal{F}_{\text{tssIBE}}$ in the Random Oracle Model. That is, for any PPT environment $\mathcal{E}$ and any PPT adversary $\mathcal{A}$ corrupting a subset of parties, there exists a PPT simulator Sim such that:*

$$\Pr[\text{REAL}_{\Pi_{\text{tssIBE}}, \mathcal{A}, \mathcal{E}}(\lambda) = 1] - \Pr[\text{IDEAL}_{\mathcal{F}_{\text{tssIBE}}, \text{Sim}, \mathcal{E}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

where $\text{negl}(\lambda)$ is a negligible function in the security parameter λ .

Proof. The proof proceeds by constructing a simulator Sim that interacts with the ideal functionality $\mathcal{F}_{\text{tsslBE}}$ and the environment $\mathcal{E}$ in the ideal world, such that no PPT environment can distinguish between the real execution of Π_{tsslBE} and the ideal execution with $\mathcal{F}_{\text{tsslBE}}$.

Consider the following simulator that interacts with $\mathcal{E}$ and $\mathcal{F}_{\text{tsslBE}}$ in the ideal world.

- On $(sid, \text{"init"}, \mathcal{S}_i)$ from $\mathcal{F}_{\text{tsslBE}}$:
If this is the first such call, Sim samples $sk_i \leftarrow \mathbb{Z}_q$ and computes $pk_i \leftarrow g_2^{sk_i}$. It stores (pk_i, sk_i) and returns pk_i to $\mathcal{F}_{\text{tsslBE}}$. Otherwise, it returns the public key pk_i stored in its state.
- On $(sid, \text{"encrypt"}, \vec{mpk}, t, id, m, aad)$ from $\mathcal{F}_{\text{tsslBE}}$:
 Sim runs the honest "encrypt" interface of Π_{tsslBE} to produce a ciphertext C and sends C back to $\mathcal{F}_{\text{tsslBE}}$.
- On $(sid, \text{"encrypt"}, \vec{mpk}, t, id, |m|, aad)$ from $\mathcal{F}_{\text{tsslBE}}$:
 Sim executes as follows:
 - sample $r \leftarrow \mathbb{Z}_q$
 - compute $nonce \leftarrow g_2^r$
 - sample $c_r \leftarrow \{0, 1\}^\lambda$
 - sample $c'_i \leftarrow \{0, 1\}^\lambda$ for $i \in [n]$
 - sample $k_{sym} \leftarrow \text{SYMENC.keygen}()$
 - compute $sem \leftarrow \text{SYMENC.encrypt}(k_{sym}, 0^{|m|}, aad)$
 - set $C \leftarrow (\vec{pk}, t, nonce, c_r, c'_{[n]}, sem)$,
 - and add (C, id, aad, r) to $CTXT$.

Finally, it sends C to $\mathcal{F}_{\text{tsslBE}}$.

- On $(sid, \text{"ekderive"}, \mathcal{S}_i, id, \mathcal{U})$ from $\mathcal{F}_{\text{tsslBE}}$ for an honest user $\mathcal{U}$:
 Sim lets $\mathcal{S}_i$, and $\mathcal{U}$ follow the "ekderive" interface on Π_{tsslBE} protocol.
- On $(sid, \text{"ekderive"}, \mathcal{S}_i, id, \mathcal{U})$ from $\mathcal{F}_{\text{tsslBE}}$ for a corrupt user $\mathcal{U}$:
 Sim receives $(sid, \text{"tpk"}, tpk)$ from $\mathcal{U}$, parses $(tpk_1, tpk_2) \leftarrow tpk$ and checks whether $e(tpk_1, g_2) = e(g_1, tpk_2)$; if not, it ignores this input. Otherwise, Sim computes $\sigma \leftarrow H_1(id)^{sk_i}$, samples $r \leftarrow \mathbb{Z}_q$, computes $es \leftarrow (g_1^r, tpk_1^r \cdot \sigma)$, and sends $(sid, \text{"encsig"}, id, tpk, es)$.

Sim goes over all $(C, id, aad, r) \in CTXT$ to

- parse $C \rightarrow (\vec{pk}, t, nonce, c_r, c'_{[n]}, sem)$,
- derive the server set $\vec{\mathcal{S}}$ corresponding to the public keys in $\vec{pk}$,
- send an input $(sid, \text{"decrypt"}, \vec{\mathcal{S}}, \vec{pk}, t, id, C, aad)$ to $\mathcal{F}_{\text{tsslBE}}$ to obtain a response $(sid, \text{"decrypt"}, C, id, m, aad)$,
- program random k_{sym} , such that $\text{SYMENC.decrypt}(k_{sym}, sem, aad) = m$
- sample $k \leftarrow \{0, 1\}^\lambda$
- sample $k_{[n]} \leftarrow \text{TSS.share}(k, n, t)$
- compute $k_r \leftarrow r \oplus c_r$
- compute $T_i \leftarrow e(H_1(id), (\vec{pk})_i)^r$ for all $i \in [n]$
- program the maps for random oracles H_2 , and H_3 as

$$\forall i \in [n] : H_2[i, (\vec{pk})_i, H_1[id], nonce, T_i] \leftarrow c'_i \oplus k_i$$

$$H_3[k, \vec{pk}, t, c'_{[n]}] \leftarrow (k_r, k_{sym})$$

- and to delete (C, id, aad, r) from $CTXT$.

Note that programming the random oracles this way always works unless Sim aborted earlier. Also note that, by programming the random oracles in this way, the "**decrypt**" interface of Π_{tssIBE} correctly decrypts C under id to m .

- On $(sid, \text{"decrypt"}, \vec{S}, \vec{pk}, t, C, id, aad)$ from $\mathcal{F}_{\text{tssIBE}}$:

Sim uses the "**decrypt**" interface on Π_{tssIBE} protocol to decrypt C under id . If successful, it sends m back to $\mathcal{F}_{\text{tssIBE}}$. Otherwise, it sends $\perp$.

co-BDH reduction. We show that a distinguisher $\mathcal{D}$ that can distinguish between the real execution of Π_{tssIBE} and the simulator Sim can be turned into an algorithm $\mathcal{B}$ that solves the co-BDH problem. On input a co-BDH challenge $(U_1, V_1, V_2, W_2) \in \mathbb{G}_1 \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_2$ with the goal of outputting $e(g_1, g_2)^{\alpha\beta\gamma}$ for hidden α, β, γ such that $U_1 = g_1^\alpha$, $V_1 = g_1^\beta$, $V_2 = g_2^\beta$, and $W_2 = g_2^\gamma$, algorithm $\mathcal{B}$ runs $\mathcal{E}$ and $\mathcal{A}$ in an environment like the one provided by Sim above, except that:

- For the init of each server $\mathcal{S}_i$, it uses $mpk_i \leftarrow V_2^{\theta_i}$ for a random $\theta_i \leftarrow \mathbb{Z}_q$.
- It simulates the random oracle $H_1(id)$, by choosing $r \leftarrow \mathbb{Z}_q$, storing $H_1[id] \leftarrow (g_1^r, r)$, and returning g_1^r , except for one randomly guessed id^* , it returns $H_1[id^*] \leftarrow U_1$ instead.
- On input $(sid, \text{"ekderive"}, \mathcal{S}_i, id, \mathcal{U})$ for any user where $id \neq id^*$:
 - $\mathcal{B}$ receives $(sid, \text{"tpk"}, tpk)$ from $\mathcal{U}$,
 - parses $(tpk_1, tpk_2) \leftarrow tpk$ and checks whether $e(tpk_1, g_2) = e(g_1, tpk_2)$; if not, it ignores this input.
 - Otherwise it looks up the stored $(g_1^r, r) \leftarrow H_1[id]$, computes $\sigma \leftarrow V_1^{r\theta_i}$, samples $r' \leftarrow \mathbb{Z}_q$, computes $es \leftarrow (g_1^{r'}, tpk_1^{r'} \cdot \sigma)$, and sends $(sid, \text{"encsig"}, id, tpk, es)$.

If $\mathcal{U}$ is corrupt, then $\mathcal{B}$ patches the ciphertexts just as Sim does.

- On input $(sid, \text{"ekderive"}, \mathcal{S}_i, id^*, \mathcal{U})$: $\mathcal{B}$ aborts if $\mathcal{U}$ is corrupt. Otherwise, it does the following:
 - sample $u \leftarrow \mathbb{Z}_q$ and computes $\tilde{tpk} \leftarrow (V_1 \cdot g_1^u, V_2 \cdot g_2^u)$,
 - send $(sid, \text{"tpk"}, \tilde{tpk})$ from $\mathcal{U}$.
 - sample $s \leftarrow \mathbb{Z}_q$, computes $es \leftarrow (g_1^s \cdot U_1^{-\theta_i}, g_1^{us} \cdot U_1^{-\theta_i u} \cdot V_1^{-\theta_i s})$,
 - send $(sid, \text{"encsig"}, id, \tilde{tpk}, es)$ to $\mathcal{F}_{\text{tssIBE}}$.
- On input $(sid, \text{"encrypt"}, \vec{mpk}, t, id, m, aad, \mathcal{P})$:

On one randomly chosen input $(sid, \text{"encrypt"}, \vec{mpk}, t, id, \ell, aad)$ from $\mathcal{F}_{\text{tssIBE}}$, it gives up if $id \neq id^*$; otherwise, it sets $C^* \leftarrow (mpk, t, nonce, c_r, c'_{[n]}, sem)$ where

- $nonce \leftarrow W_2$,
- $c_r \leftarrow \{0, 1\}^\kappa$ is a uniform κ -bit string,
- $\forall i \in [n] : c'_i \leftarrow \{0, 1\}^\kappa$ independent uniform strings,
- $sem \leftarrow \{0, 1\}^\ell$ an independent uniform string of length ℓ ,

adds (C^*, id^*, aad, s) to $CTXT$ for $s \leftarrow \{0, 1\}^\kappa$, and responds C^* to $\mathcal{F}_{\text{tssIBE}}$.

For all other inputs, $\mathcal{B}$ follows the same strategy as Sim does.

- On $(sid, \text{"decrypt"}, \vec{mpk}, t, C = (mpk, t, nonce, c_r, c'_{[n]}, sem), id, aad)$ from $\mathcal{F}_{\text{tssIBE}}$, $\mathcal{B}$ does not decrypt using any secret key, but instead checks whether there exist $((k, \vec{pk}, t, c'_{[n]}), (k_r, k_{sym})) \in H_3$, such that $g_2^{k_r \oplus c_r} = nonce$. Return $\perp$ if no such entry. If so, then it computes $r \leftarrow c_r \oplus k_r$, decrypts all the encrypted shares with r and then checks if the key shares form a valid secret sharing of k . Otherwise, it returns $\perp$. If all checks pass, it returns $m \leftarrow \text{SYMENC.decrypt}(k_{sym}, sem, aad)$ to $\mathcal{F}_{\text{tssIBE}}$; if not, or if no such entry was found, it returns $m \leftarrow \perp$ to $\mathcal{F}_{\text{tssIBE}}$.

The only way that this decryption method produces a different outcome than the real decryption of Π_{tslBE} is if, during decryption, no entry $((k, \vec{mpk}, t, c'_{[n]}), (k_r, k_{\text{sym}}))$ exists in H_3 , but later a new random-oracle query to H_3 on $(k, \vec{mpk}, t, c'_{[n]})$ creates exactly such an entry with $g_2^{k_r \oplus c_r} = \text{nonce}$. With at most q_D decryption queries and q_H random-oracle queries, this happens with probability at most $q_H q_D / 2^\kappa$.

To cause the original simulator Sim to abort, $\mathcal{A}$ must make a query to H_2 so that there exists $(C, id, aad, r) \in CTXT$ with

$$H_2[i^*, (\vec{mpk})_{i^*}, H_1[id], \text{nonce}, e(H_1(id), (\vec{mpk})_{i^*})^r] \text{ is queried.}$$

Algorithm $\mathcal{B}$'s strategy is that the offending query occurs for the tuple $(C^*, id^*, aad, s) \in CTXT$, in which case we would have that

$$T^{1/\theta_i} := e(H_1(id^*), (\vec{mpk})_i)^{\log_{g_2} \text{nonce}} = e(U_1, V_2)^{\log_{g_2} W_2} = e(g_1, g_2)^{\alpha\beta\gamma},$$

which is the solution to $\mathcal{B}$'s co-BDH challenge. Since $\mathcal{B}$ cannot test which random-oracle query is the offending one, it simply outputs the key of a randomly chosen entry, scaled by its corresponding θ_i , among its H_2 queries, giving it a probability of at least $1/q_H$ to guess correctly. That query also has to trigger abortion for the tuple $(C^*, id^*, aad, s) \in CTXT$, however, which happens with probability $1/q_E$ if $\mathcal{B}$ didn't give up prematurely.

There are two reasons that cause $\mathcal{B}$ to give up prematurely. The first is if an honest server $\mathcal{S}_i$ calls $\mathcal{F}_{\text{tslBE}}$ with $(sid, \text{"ekderive"}, \mathcal{S}_i, id^*, \mathcal{U})$ for a corrupt $\mathcal{U}$. The other reason for $\mathcal{B}$ to give up early is if the randomly chosen "encrypt" input is for $id \neq id^*$. This can indeed happen, but since $\mathcal{A}$'s view is independent of $\mathcal{B}$'s choice of id^* as long as it doesn't give up, we have a probability at least $1/q_H$ that $id = id^*$.

Overall, $\mathcal{B}$ therefore outputs the solution to the co-BDH problem with probability at least $1/(q_E \cdot q_H^2)$. $\square$

On the symenc scheme. For the UC proof we require programmability of the symmetric encryption scheme. As described in Section 5.2.1, Seal offers two different symmetric encryption schemes: HMAC-CTR-based and AES-GCM-based. The HMAC-CTR-based scheme can be written succinctly as:

$$\text{sem} \leftarrow \text{SYMENC.encrypt}(k, M = m_{[l]}, aad) = (c_i = m_i \oplus H(k, \text{"enc"}|i))_{i \in [l]}, H(k, \text{"mac"}|aad|c_{[l]}))$$

To program this to decrypt to $M' = m'_{[l]}$ under an independently random key k' , we program the RO as follows:

$$\begin{aligned} H(k', \text{"enc"}|i) &\leftarrow m'_{[i]} \oplus m_{[i]} \oplus H(k, \text{"enc"}|i) \\ H(k', \text{"mac"}|aad|c_{[l]}) &\leftarrow H(k, \text{"mac"}|aad|c_{[l]}) \end{aligned}$$

For the AES-GCM-based scheme, we are not able to prove UC security due to the lack of programmability. To maintain UC security, the simulator must be able to equivocate: it must retroactively program the symmetric encryption scheme so that the previously simulated ciphertext C decrypts to the real message m under the newly revealed key material. In contrast, a game-based security definition can avoid this programmability requirement by imposing restrictions on the adversary's behavior. Specifically, we can define security so that the adversary *loses* the security game if it ever requests an extracted key for an identity id that was used in any encryption challenge query. This restriction is natural in the context of identity-based encryption: it corresponds to the standard adaptive chosen ciphertext attack (IND-ID-CCA) security notion, where the adversary cannot query the key extraction oracle for the challenge identity. Under such a definition, the simulator never needs to equivocate simulated ciphertexts, because the adversary is prevented from obtaining keys that would allow decryption of challenge ciphertexts. This approach provides a more practical security guarantee for schemes like AES-GCM that do not admit efficient programmability, at the cost of a weaker (but still standard) security model compared to full UC security.

5.5 Share Consistency Guarantee

Seal allows decryption of ciphertexts with varying sets of decryption keys. The TSSIBE-SHARECON property ensures that the decryption result is the same regardless of which set is used, including adversarial ones.

Definition 6 (TSSIBE-SHARECON). *Adversary has negligible advantage in the following game with a challenger.*

1. Receive $pk_{[n]}$ from the Adversary
2. Receive $ID, c, \vec{sk}_{ID}, \vec{sk}'_{ID}$ from Adversary
3. Adversary wins if
 - (a) $\vec{sk}_{ID} \neq \vec{sk}'_{ID}$ and both have exactly t entries which are not $\perp$.
 - (b) $\text{admissible}(\vec{sk}_{ID}, \vec{pk}, ID, t) = \text{admissible}(\vec{sk}'_{ID}, \vec{pk}, ID, t) = 1$.
 - (c) $\text{decrypt}(\vec{sk}_{ID}, c, \text{aad}, ID) \neq \text{decrypt}(\vec{sk}'_{ID}, c, \text{aad}, ID)$, including the case that one of these is $\perp$.

Theorem 4. *TSS-BF-KEM unconditionally satisfies TSSIBE-SHARECON.*

Proof. If both decryptions fail then the result holds. Now assume the decryption with $\vec{sk}_{ID}$ returns M . Roughly, the intuition is that the *nonce* unconditionally binds to r , and given that value, the encapsulated key is deterministically derived.

Formally, Let the ciphertext be

$$c = (\vec{pk}, t, \text{nonce}, c_r, c_{[n]}, \text{sem}),$$

Assume the decryption using vector $\vec{sk}_{ID}$, succeeds and outputs M . We show that decryption with any other admissible vector $\vec{sk}'_{ID}$, must also succeed and output the same M .

Since decryption with vector $\vec{sk}_{ID}$ succeeds, this implies:

1. Each share k_i , such that $\vec{sk}_{ID,i} \neq \perp$ is recovered as:

$$\begin{aligned} k_i &= c_i \oplus H_2(i, (\vec{pk})_i, H_1(ID), \text{nonce}, e(sk_{ID,i}, \text{nonce})) \\ &= c_i \oplus H_2(i, (\vec{pk})_i, H_1(ID), \text{nonce}, e(H_1(ID), \text{pk}_i)^r). \end{aligned}$$

2. The t shares k_i interpolate the unique degree- $(t-1)$ polynomial $p(x)$, yielding $k = p(0)$.

3. Let $(k_r, k_{sym}) \leftarrow H_3(k, \vec{pk}, t, c'_{[n]})$

4. Randomness r is recovered via:

$$r = c_r \oplus k_r.$$

5. The nonce check $g_2^r = \text{nonce}$ passes.

6. For all j , such that $\vec{sk}_{ID,j} = \perp$, we have:

$$\begin{aligned} k_j &= c_j \oplus H_2(j, (\vec{pk})_j, H_1(ID), \text{nonce}, e(H_1(ID), \text{pk}_j^r)) \\ &= c_j \oplus H_2(j, (\vec{pk})_j, H_1(ID), \text{nonce}, e(sk_{ID,j}, \text{nonce})) \end{aligned}$$

7. By the share consistency check, it is guaranteed that these k_j 's are same as those uniquely determined by k_i 's. Therefore they interpolate the same k .

8. The payload sem is successfully decrypted with k_{sym} to yield M .

Now we analyze decryption with the vector $\vec{sk}'_{ID}$:

1. For each j , such that $\vec{sk}'_{ID,j} \neq \perp$, compute:

$$\begin{aligned} k_j &= c_j \oplus H_2(j, (\vec{pk})_j, H_1(ID), \text{nonce}, e(sk_{ID,j}, \text{nonce})) \\ &= c_j \oplus H_2(j, (\vec{pk})_j, H_1(ID), \text{nonce}, e(H_1(ID), \text{pk}_j^r)). \end{aligned}$$

which is same as those computed in Step 6 above.

2. The t shares k_j interpolate the same k as above, since share consistency passed for $\vec{s}k_{ID}$.
3. Randomness r is same as above, since k_r is same. So nonce check passes. In addition, share consistency also passes, since it passed for $\vec{s}k_{ID}$ with the same r .
4. The authenticated encryption is decrypted to recover the same message M , since k_{sym} is same.

Therefore, if decryption for the vector $\vec{s}k_{ID}$ passes and outputs M , so does for vector $\vec{s}k'_{ID}$. $\square$

5.6 Comparison to vetKeys

Our core cryptography is similar to vetKeys [CCN⁺23], but differs in specific details of IBE instantiation, threshold sharing, targeting multiple independent IBE keys, and has a different security analysis.

An important distinction is the way we handle threshold decryption. We allow the threshold servers to generate their IBE keys independently, rather than secret share those with the help of a setup DKG, which increases operational complexity.

- While vetKeys uses a setup DKG which results in a single encryption public key, we allow independent public keys for servers.
- We follow a KEM-DEM structure for the encryption, where a symmetric key is threshold shared and encrypted to the independent public keys. The message itself is symmetric encrypted using the key.
- We batch the independent encryptions to use common randomness and achieve $\approx 50\%$ reduction in KEM size. However, as opposed to constant size ciphertexts in vetKeys, our KEM size still scales linearly with the number of servers as a cost for key independence.
- An important security consideration in our model is **share consistency**: in on-chain scenarios public keys and secret keys can be adversarially generated, and the adversary can also dictate which subset of secret keys to use for decryption in a threshold setting. We ensure that the result of decryption is the same, including its failure, regardless of which subset is chosen. This property is essential for maintaining deterministic behavior in adversarial environments.

6 MPC Committee for Seal Key Service

Section 2 noted that a Seal key server may be realized as an MPC committee. We instantiate that option here: a committee jointly holds a single BLS signing key and produces the encrypted BLS signatures of Section 5.1.1 in threshold form, so clients keep encrypting to one committee public key vk as in Section 5, oblivious to the committee behind it. We work in the synchronous network setting.

6.1 Network Model

We assume a synchronous network with a known upper bound Δ on message delivery time. The protocol operates in discrete rounds, where each round has duration Δ .

We assume n parties with threshold t for reconstruction, where at most $t - 1$ parties may be malicious with regards to privacy. During DKG and key rotation, the protocol completes only if *all* parties from the current committee cooperate, and otherwise aborts while identifying the cheater (including ones that did not respond in time).

PKI and Coordinator Model: We assume a PKI where each party P_i has:

- An encryption key pair (pk_i^{enc}, sk_i^{enc}) for receiving encrypted shares.
- A signature key pair (pk_i^{sig}, sk_i^{sig}) for authenticating messages.

The protocol uses a designated *coordinator* (which may be one of the parties or a separate entity) to relay messages. Instead of direct broadcast, parties send signed messages to the coordinator, who collects and distributes them to all parties. The coordinator is trusted for liveness (message delivery)

but not for correctness—any misbehavior by the coordinator can be detected by parties through signature verification and consistency checks.

To ensure that all parties agree on the same *finalization* view—and not solely on whatever dealer messages the coordinator may have relayed—each party publishes its signed verification message (joint verification key vk and aggregated VSS keys) to a *Total Order Broadcast (TOB)* channel, so every party observes the same ordered sequence of contributions.

Multi-Recipient Encryption: The protocol uses a multi-recipient encryption scheme MREC from the Multi-Recipient Encryption definition in Section 7. We write algorithms as MREC.keygen , MREC.enc , etc. here only for readability, and we denote ciphertexts by E (e.g. dealer ciphertexts E_i , E_j in Phase B), matching that section.

The tag tag is the ADO-CCA associated data in that definition: the random-oracle inputs (e.g. $H(\text{tag}, i, \cdot)$ and $H_2(\text{tag}, c)$ where c is the g_2^T -component of E) bind the ciphertext to tag , so encryptions under different tags are separate security instances even if an adversary obtains reveal queries under other tags. In Phase B below, dealer P_j calls enc with $\text{tag} = j$, and every dec , reveal , and verify on ciphertext E_j uses the same tag j . The MEGa of [GS22] is stated without an explicit tag; our definition makes the associated-data field explicit for ADO-CCA (Section 7).

Syntax (as in Section 7).

- $\text{MREC.keygen}(1^\lambda) \rightarrow (\text{sk}, \text{pk})$: Per-user key generation.
- $\text{MREC.enc}(PK, M, \text{tag}) \rightarrow E$: On public-key sequence $PK = \langle \text{pk}_i \rangle_{i=1}^n$ and plaintext sequence $M = \langle m_i \rangle_{i=1}^n$, output ciphertext E .
- $\text{MREC.dec}(\text{sk}_i, i, E, \text{tag}) \rightarrow m$: Return the i -th plaintext or failure; when party P_i decrypts we denote this value by m_i .
- $\text{MREC.reveal}(\text{sk}_i, i, E, \text{tag}) \rightarrow (m, \pi)$: Return the same i -th plaintext m as dec and a publicly verifiable proof π .
- $\text{MREC.verify}(\text{pk}_i, m, \pi, i, E, \text{tag}) \rightarrow \{0, 1\}$: Verify a reveal for index i ; the argument m is the claimed i -th plaintext (as output by dec/reveal).

6.2 Protocol

The protocol is organized in three phases: **(A) Registration**: the coordinator asks all members to register; each party registers pk_i^{enc} and pk_i^{sig} (in practice this may be done onchain, in which case the coordinator sees all registrations and a separate consistency check on the message set may be redundant). **(B) Message creation**: the coordinator, having seen all registrations (e.g. onchain), asks all members to create their dealer message; note that each party acts both as dealer and as receiver. **(C) Finalization**: the coordinator collects all dealer messages, sends them to every member, and all members process the received messages and finalize (compute vk , aggregated VSS keys, and consistency hash; then verify via TOB and output). The formal description below specifies Phase B (message creation) and Phase C (finalization) in sequence; Phase A is assumed complete.

Formal description:

1. Phase B. Message creation (each party P_i as dealer):

- (a) Sample a uniformly random degree- $t-1$ polynomial $p_i(X) = c_{i,0} + c_{i,1}X + \dots + c_{i,t-1}X^{t-1}$ from $\mathbb{Z}_q[X]$.
- (b) Compute $C_{i,k} \leftarrow g^{c_{i,k}}$ for $k \in [0, t-1]$.
- (c) Compute a non-interactive PoK π_i for $c_{i,0}$: sample $r_i \leftarrow \mathbb{Z}_q$, compute $R_i = g^{r_i}$, $c_i = H(C_{i,0} \| R_i \| i)$, and $z_i = r_i + c_i \cdot c_{i,0}$, where H is a hash function. The proof is $\pi_i = (R_i, z_i)$.
- (d) Compute $s_{i,j} \leftarrow p_i(j)$ for $j \in [1, n]$.
- (e) Sample $E_i \leftarrow \text{MREC.enc}(PK, S_i, i)$ where $PK = (\text{pk}_1^{\text{enc}}, \dots, \text{pk}_n^{\text{enc}})$ and $S_i = (s_{i,1}, \dots, s_{i,n})$.
- (f) Send signed message $(C_{i,0}, \dots, C_{i,t-1}, E_i, \pi_i)$ to the coordinator.

2. Phase C. Finalization (each party P_i as receiver, then all finalize):

- (a) Receive all n signed dealer messages from the coordinator. Verify all signatures.
- (b) For each dealer P_j , $j \in [n] \setminus \{i\}$, parse $(C_{j,0}, \dots, C_{j,t-1}, E_j, \pi_j)$ and do:
 - i. Sanity checks: check message length, group membership, and that all elements are well-formed.
 - ii. Verify the PoK $\pi_j = (R_j, z_j)$: check $g^{z_j} = R_j \cdot C_{j,0}^{c_j}$ where $c_j = H(C_{j,0} \| R_j \| j)$. If verification fails, abort.
 - iii. Decrypt $s_{j,i} \leftarrow \text{MREC.dec}(\text{sk}_i^{\text{enc}}, i, E_j, j)$.
 - iv. Verify $g^{s_{j,i}} = \prod_{k=0}^{t-1} C_{j,k}^{i^k}$. If not, compute $F_{i,j} = \text{MREC.reveal}(\text{sk}_i^{\text{enc}}, i, E_j, j)$, send signed complaint to the coordinator, and abort.
- (c) Compute $\text{vk} = \prod_{j \in [n]} C_{j,0}$ as the joint verification key.
- (d) Compute aggregated VSS verification keys: $\{C_k^{\text{agg}}\}_{k=0}^{t-1} = \{\prod_{j \in [n]} C_{j,k}\}_{k=0}^{t-1}$.
- (e) Compute consistency hash: $h_i = H(\text{all received messages})$.
- (f) Send signed $(\text{vk}, \{C_k^{\text{agg}}\}_{k=0}^{t-1}, h_i)$ to the TOB. (An equivalent option is to send signed *evaluation points* instead, from which vk and $\{C_k^{\text{agg}}\}$ can be derived; the same consistency check then applies.)
- (g) Private output $s_i = \sum_{j \in [n]} s_{j,i}$ as P_i 's secret share of vk .

3. Phase C (continued). TOB and consistency:

- (a) Obtain all signed verification messages $\{(\text{vk}_j, \{C_k^{\text{agg},j}\}, h_j)\}_{j \in [n]}$ by following the TOB channel: each party publishes its signed $(\text{vk}, \{C_k^{\text{agg}}\}, h_i)$ to the TOB, and every party waits until exactly n such messages have been delivered in total order. Verify all signatures (restart without non-responding parties if needed).
- (b) If any complaint $(F_{i,j}, \sigma)$ is received from the coordinator, verify:
 - i. Parse $F_{i,j} = (s_{j,i}, \pi)$ where π is the reveal proof.
 - ii. Check the signature σ from complainer P_i .
 - iii. Verify the reveal proof: $\text{MREC.verify}(\text{pk}_i^{\text{enc}}, s_{j,i}, \pi, i, E_j, j) = 1$.
 - iv. Check that the revealed share is inconsistent with commitments: $g^{s_{j,i}} \neq \prod_{k=0}^{t-1} C_{j,k}^{i^k}$.

If both the reveal proof verifies and the share is inconsistent with commitments, dealer P_j cheated—abort and exclude P_j . If the reveal proof fails, complainer P_i submitted an invalid complaint. Anyone can perform this verification using public data.
- (c) Verify $h_j = h_i$ for all j . If any hash differs, abort (coordinator misbehavior detected).
- (d) Verify vk_j and $\{C_k^{\text{agg},j}\}$ are identical for all j , otherwise abort.
- (e) Output success.

TOB and coordinator consistency. Phase C has each party compute $(\text{vk}, \{C_k^{\text{agg}}\}_{k=0}^{t-1}, h_i)$ from the dealer messages it received and post a signed verification message to the TOB; parties wait until exactly n such messages have been delivered in total order (see the final enumerated item, *Phase C (continued) — TOB and consistency*, above). That guarantees every honest party reasons about the same multiset of verification messages, rather than about coordinator-dependent views of who said what. Parties then require that all posted tuples agree on vk , on the aggregated VSS keys $\{C_k^{\text{agg}}\}$, and on the consistency hash h ; the same checks can be implemented onchain if the TOB is instantiated there. If the coordinator had forwarded inconsistent sets of dealer messages to different parties, those local computations would diverge, so the posted vk_j , $\{C_k^{\text{agg},j}\}$, or h_j would not all match and the protocol aborts.

6.3 Key Rotation and Committee Change Protocol

We now describe the key rotation protocol that allows the MPC committee to migrate from an old committee of size n with threshold t to a new committee of size n' with threshold t' , while keeping the same joint verification key vk . This is useful for adapting to changing security requirements, committee

composition, or party availability. The key property is that the joint verification key $\text{vk} = g_2^{sk}$ remains unchanged, but parties in the new committee obtain fresh shares of the same secret sk under the new threshold t' . An important consequence is that *clients can continue to encrypt to the same public key vk without any change*; they do not need to be aware of the underlying threshold committee or its rotation. As in the DKG, we use the coordinator model with PKI and the TOB for finalization.

The same three-phase structure applies: **(A) Registration**—new committee members register their keys (e.g. onchain). **(B) Message creation**—only *old* committee members create resharing messages; the coordinator, having seen registrations, asks them to create and send signed resharing messages. **(C) Finalization**—the coordinator distributes the collected messages to all (new) members, and all new members process and finalize via the TOB as in DKG.

Prerequisites:

- Old committee: Parties $\mathbb{P}^{old} = \{P_1^{old}, \dots, P_n^{old}\}$ with threshold t
- New committee: Parties $\mathbb{P}^{new} = \{P_1^{new}, \dots, P_{n'}^{new}\}$ with threshold t' (where $t' \leq n'$)
- Existing joint verification key: $\text{vk} = g_2^{sk}$
- Each old party P_i^{old} holds secret share: sk_i^{old} such that $sk = \sum_{j \in \mathcal{I}} \lambda_{0,j,\mathcal{I}} sk_j^{old}$ for any set $\mathcal{I} \subseteq \mathbb{P}^{old}$ of size t
- Old VSS verification keys: $\{C_k^{old}\}_{k=0}^{t-1}$
- Old committee PKI: Each P_j^{old} has keys $((\text{pk}_j^{old})^{enc}, (\text{pk}_j^{old})^{sig})$
- New committee PKI: Each P_j^{new} has keys $((\text{pk}_j^{new})^{enc}, (\text{pk}_j^{new})^{sig})$
- Requirement: At least t parties from the old committee participate (to reconstruct the secret). That is:

$$|\mathbb{P}_{new} \cap \mathbb{P}_{old}| \geq t$$

Formal description:

1. Phase B. Message creation (old committee dealers P_i^{old}):

- Sample a uniformly random degree- $(t'-1)$ polynomial $p_i(X) = d_{i,0} + d_{i,1}X + \dots + d_{i,t'-1}X^{t'-1}$ from $\mathbb{Z}_q[X]$ with the constraint:

$$d_{i,0} = sk_i^{old}$$

This ensures the constant term is the party's old share.

- Compute $D_{i,k} \leftarrow g^{d_{i,k}}$ for $k \in [0, t' - 1]$.
- Compute new shares for the new committee: $s'_{i,j} \leftarrow p_i(j)$ for $j \in [1, n']$.
- Sample $E'_i \leftarrow \text{MREC.enc}(PK^{new}, S'_i, i)$ where $PK^{new} = ((\text{pk}_1^{new})^{enc}, \dots, (\text{pk}_{n'}^{new})^{enc})$ and $S'_i = (s'_{i,1}, \dots, s'_{i,n'})$.
- Send signed message $(D_{i,0}, \dots, D_{i,t'-1}, E'_i)$ to coordinator.

2. Phase C. Finalization (new committee members P_i^{new} : process messages, then TOB and consistency):

- Receive signed resharing messages from coordinator. Verify all signatures. Let $\mathcal{D}$ denote the set of t old dealers whose messages were received.
- For each dealer $P_j^{old} \in \mathcal{D}$, parse $(D_{j,0}, \dots, D_{j,t'-1}, E'_j)$ and do:
 - Sanity checks: verify message format, group membership, and validity of encryptions.
 - Verify consistency with old share: $D_{j,0} = \prod_{k=0}^{t-1} (C_k^{old})^{j^k}$. If not, abort.
 - Decrypt $s'_{j,i} \leftarrow \text{MREC.dec}((sk_i^{new})^{enc}, i, E'_j, j)$.
 - Verify $g^{s'_{j,i}} = \prod_{k=0}^{t'-1} D_{j,k}^{i^k}$. If not, compute $F_{i,j} = \text{MREC.reveal}((sk_i^{new})^{enc}, i, E'_j, j)$, send signed complaint to coordinator, and abort.

- (c) Compute the new joint verification key using Lagrange interpolation:

$$\mathbf{vk}' = \prod_{j \in \mathcal{D}} D_{j,0}^{\lambda_{0,j,\mathcal{D}}} = g^{sk} = \mathbf{vk}$$

where $\lambda_{0,j,\mathcal{D}} = \prod_{\ell \in \mathcal{D} \setminus \{j\}} \frac{\ell}{\ell - j}$ are Lagrange coefficients. Here j and ℓ denote the *old committee* indices (the index of the dealer in $\mathbb{P}^{old}$), not their index in the new committee; in implementations a mapping from sender to old index is used.

- (d) Compute new aggregated VSS verification keys: $\{(C'_k)^{agg}\}_{k=0}^{t'-1} = \left\{ \prod_{j \in \mathcal{D}} D_{j,k}^{\lambda_{0,j,\mathcal{D}}} \right\}_{k=0}^{t'-1}$, where again j runs over old committee indices in $\mathcal{D}$.
- (e) Compute consistency hash: $h'_i = H(\text{all received messages})$.
- (f) Send signed $(\mathbf{vk}', \{(C'_k)^{agg}\}, h'_i)$ to the TOB. (Equivalently, one may send signed evaluation points instead, from which $\mathbf{vk}'$ and $\{(C'_k)^{agg}\}$ can be derived.)
- (g) Private output: $sk_i^{new} = \sum_{j \in \mathcal{D}} \lambda_{0,j,\mathcal{D}} \cdot s'_{j,i}$.

3. Phase C (continued). TOB and consistency:

- (a) Obtain all signed verification messages $\{(\mathbf{vk}'_j, \{(C'_k)^{agg,j}\}, h'_j)\}_{j \in [n']}$ by following the TOB channel: each new committee member publishes its signed $(\mathbf{vk}', \{(C'_k)^{agg}\}, h'_i)$ to the TOB, and every party waits until exactly n' such messages have been delivered in total order. Verify all signatures.
- (b) If any complaint $(F_{i,j}, \sigma)$ is received from the coordinator, verify:
- Parse $F_{i,j} = (s'_{j,i}, \pi)$ where π is the reveal proof.
 - Check the signature σ from complainer P_i^{new} .
 - Verify the reveal proof: $\text{MREC.verify}((\text{pk}_i^{new})^{enc}, s'_{j,i}, \pi, i, E'_j, j) = 1$.
 - Check that the revealed share is inconsistent with commitments: $g^{s'_{j,i}} \neq \prod_{k=0}^{t'-1} D_{j,k}^{i^k}$.

If both the reveal proof verifies and the share is inconsistent with commitments, dealer P_j^{old} cheated—abort and exclude P_j^{old} . If the reveal proof fails, complainer P_i^{new} submitted an invalid complaint. Anyone can perform this verification using public data.

- (c) Verify $h'_j = h'_i$ for all j . If any hash differs, abort (coordinator misbehavior detected).
- (d) Verify $\mathbf{vk}'_j$ and $\{(C'_k)^{agg,j}\}$ are identical for all j , otherwise abort.
- (e) Verify $\mathbf{vk}' = \mathbf{vk}$ (joint key unchanged).
- (f) Update committee parameters: $n \leftarrow n', t \leftarrow t'$.
- (g) New committee members store their secret shares: $sk_i \leftarrow sk_i^{new}$.
- (h) Old committee members can safely delete their old shares sk_i^{old} (the secret is now protected by the new committee).
- (i) Output success.

6.4 Distributed Encrypted BLS Signing

The MPC protocol described in Section 6 generates a shared BLS signing key distributed among n *key servers*. We now describe how this distributed key is used to produce encrypted BLS signatures, as defined in Section 5.1.1, in a threshold manner.

Protocol overview. The flow involves the client, an *aggregator*, and the key servers. The aggregator is a separate entity from the key servers: it receives the client's ephemeral public key and identity, fans out the signing request to the key servers, collects the encrypted signature shares from the key servers, and returns the aggregated encrypted signature to the client. The aggregator need not be trusted for correctness or privacy (it does not see secret keys or plaintext signatures); it can be implemented as a dedicated server or as part of the client. Concretely: the client sends (ephpk, ID) to the aggregator; the aggregator forwards the request to the key servers; each key server P_i responds with an encrypted

signature share to the aggregator; the aggregator verifies and combines the shares and sends the result to the client; the client decrypts to obtain the BLS signature.

Formal description. To obtain an encrypted BLS signature on identity ID , the client, aggregator, and key servers execute the following protocol:

1. **User (client) generates ephemeral key:**

- Sample $ephsk = x \leftarrow \mathbb{Z}_q$
- Compute $ephpk = (ephpk_1, ephpk_2) = (g_1^x, g_2^x)$.
- Send $(ephpk, ID)$ to the aggregator (which forwards to the key servers).

2. **Each key server P_i computes encrypted signature share:**

- Sample $r_i \leftarrow \mathbb{Z}_q$
- Compute $encsig_i = (encsig_{i,1}, encsig_{i,2}) = (g_1^{r_i}, ephpk_1^{r_i} \cdot H(ID)^{sk_i})$
- Send $encsig_i$ to the aggregator (or directly to the client if the aggregator is the client).

3. **Aggregator verifies each share:**

For each received $encsig_i$ from key server P_i , check:

$$e(encsig_{i,2}, g_2) = e(encsig_{i,1}, ephpk_2) \cdot e(H(ID), pk_i)$$

This is the **encverify** check from Section 5.1.1, ensuring key server P_i computed a valid encrypted signature share.

4. **Aggregator interpolates shares:**

Collect at least $f + 1$ verified shares from the key servers. Aggregate the encrypted signature shares homomorphically:

$$encsig = \left(\prod_{i \in \mathcal{I}} encsig_{i,1}^{\lambda_i}, \prod_{i \in \mathcal{I}} encsig_{i,2}^{\lambda_i} \right)$$

This yields:

$$encsig = (g_1^r, ephpk_1^r \cdot H(ID)^{sk})$$

where $r = \sum_{i \in \mathcal{I}} \lambda_i r_i$ and $sk = \sum_{i \in \mathcal{I}} \lambda_i sk_i$ is the reconstructed shared secret.

5. **User decrypts to obtain BLS signature:**

Using **decrypt** from Section 5.1.1:

$$sig = \frac{encsig_2}{(encsig_1)^{ephsk}} = \frac{ephpk_1^r \cdot H(ID)^{sk}}{(g_1^r)^x} = \frac{g_1^{rx} \cdot H(ID)^{sk}}{g_1^{rx}} = H(ID)^{sk}$$

6. **User verifies the signature:**

Check that $e(sig, g_2) = e(H(ID), vk)$, which confirms $sig = H(ID)^{sk}$ is a valid BLS signature under the joint verification key vk .

6.5 Security of the Distributed Encrypted BLS Signing Protocol

We now prove the security of the distributed encrypted BLS signing protocol. Our security analysis covers the end-to-end security of the DKG, key rotation, and distributed encrypted BLS signing components.

6.5.1 Formal Security Definitions

We follow the security definitions from the literature on DKG for threshold BLS signatures, adapted to our synchronous dealer-receiver model.

Definition 7 (DKG Consistency and Correctness). *Let Π_{DKG} be a DKG protocol executed between n parties with threshold t , where at most $t - 1$ parties may be corrupted. We require Π_{DKG} to satisfy:*

1. **Consistency:** *If any honest party outputs $(vk_1, \dots, vk_n)$, eventually all honest parties output $(vk_1, \dots, vk_n)$.*
2. **Correctness:** *Let $\mathcal{S}$ be the set of honest parties that output $s_i \neq \perp$. Then $|\mathcal{S}| \geq t$, and there exists a degree- $(t - 1)$ polynomial $p(\cdot)$ such that $s_i = p(i)$ for all $i \in \mathcal{S}$, and $vk_i = g^{p(i)}$ for $i = 1, \dots, n$.*

These conditions guarantee that at least t honest parties receive valid shares to perform threshold operations, and that all parties agree on the verification keys.

Definition 8 (Key Rotation Consistency and Correctness). *Let Π_{Rot} be a key rotation protocol from old committee with (t, n) threshold to new committee with (t', n') threshold, where at most $t - 1$ old parties and $t' - 1$ new parties may be corrupted. We require:*

1. **Key Preservation:** *All honest new committee members output the same joint verification key $vk' = vk$.*
2. **Correctness:** *Let $\mathcal{S}'$ be the set of new committee members that output $s_i^{new} \neq \perp$. Then $|\mathcal{S}'| \geq t'$, and there exists a degree- $(t' - 1)$ polynomial $p'(\cdot)$ such that $s_i^{new} = p'(i)$ for all $i \in \mathcal{S}'$, and $vk_i = g^{p'(i)}$, where $p'(0) = p(0)$ (same secret as before rotation).*

Definition 9 (mEUF-CMA for Threshold Encrypted BLS). *Let $(\Pi_{DKG}, \Pi_{Rot}, \mathbf{ephkey}, \mathbf{encsign}, \mathbf{encverify}, \mathbf{decrypt}, \mathbf{verify})$ be a threshold encrypted BLS scheme for n parties, where Π_{DKG} satisfies consistency and correctness (Definition 7) and Π_{Rot} satisfies key rotation consistency and correctness (Definition 8).*

Define the experiment Exp^{mEUF} for a static adversary $\mathcal{A}$ controlling at most $t - 1$ parties initially:

1. **DKG:** *Execute Π_{DKG} between n parties (including $\mathcal{A}$'s corrupted parties). Denote by s_i the secret shares and vk the joint verification key output by the parties. Initialize current committee parameters $(n_{cur}, t_{cur}) = (n, t)$.*
2. **Key Rotation Queries:** *$\mathcal{A}$ can query $\text{Rotate}(n', t', \mathcal{C}')$ where:*
 - n' is the new committee size, t' is the new threshold
 - $\mathcal{C}' \subseteq [n']$ is the set of at most $t' - 1$ parties controlled by $\mathcal{A}$ in the new committee
 - Challenger executes Π_{Rot} to migrate from current committee (n_{cur}, t_{cur}) to new committee (n', t')
 - Honest parties obtain new shares s_i^{new} for the same joint verification key vk
 - Update $(n_{cur}, t_{cur}) \leftarrow (n', t')$ and corrupted set to $\mathcal{C}'$
3. **EphKey Queries:** *$\mathcal{A}$ can query $\text{EphKey}(\lambda)$. Challenger returns $\mathbf{ephpk}$ and stores it in list $\mathcal{E}_k$.*
4. **Signing Queries:** *$\mathcal{A}$ can ask all honest parties in the current committee to sign on any $(\mathbf{ephpk}, ID)$. Honest parties with $s_i \neq \perp$ return $\mathbf{encsign}(s_i, \mathbf{ephpk}, ID)$.*
5. **Output:** *$\mathcal{A}$ outputs (ID^*, sig^*) .*
6. **Winning Condition:** *$\mathcal{A}$ wins if:*
 - $e(\text{sig}^*, g_2) = e(H(ID^*), vk)$ (valid BLS signature under the joint verification key)
 - For all $\mathbf{encsign}(s_i, \mathbf{ephpk}, ID^*)$ queries made during any committee epoch, $\mathbf{ephpk} \in \mathcal{E}_k$ (honest ephemeral keys only)

We say the scheme is mEUF-CMA secure if for any PPT adversary $\mathcal{A}$, $\Pr[\text{Exp}_{\mathcal{A}, n}^{mEUF} = 1]$ is negligible.

6.5.2 Security Proofs

Theorem 5 (DKG Consistency and Correctness). *The DKG protocol from Section 6 satisfies consistency and correctness (Definition 7), if we always have $n - f \geq t$ and $t \geq f + 1$.*

Proof. Consistency: All messages are posted to the broadcast channel and are eventually visible to all parties. All aborts are done based on publicly verifiable data (commitments $C_{j,k}$ and complaint resolutions). In the synchronous model, all honest parties receive messages within the same time bounds, thus they compute identical qualified sets and output the same verification keys $(vk_1, \dots, vk_n)$.

Correctness: The protocol requires all n parties to participate. Each honest party P_i computes $s_i = \sum_{j \in [n]} s_{j,i}$ where $s_{j,i}$ is verified against commitments $C_{j,k}$ via:

$$g^{s_{j,i}} = \prod_{k=0}^{t-1} C_{j,k}^{r_k^k}$$

This ensures that the shares lie on a degree- $(t-1)$ polynomial. The complaint mechanism detects and handles invalid shares through **MREC.reveal**.

To show $|\mathcal{S}| \geq t$: The definition requires that at least t honest parties output valid shares. Let f be the number of corrupted parties, where $f \leq t-1$. There are at least $n-f$ honest parties.

The protocol assumes that it is repeated until no fraud proofs are posted. When an honest party receives an invalid share from an adversarial dealer, it posts a fraud proof using **MREC.reveal** (as specified in Phase 2 of the DKG protocol). The protocol then restarts, and this process continues until all shares are valid or adversarial dealers are excluded.

The key observation is that honest dealers always send valid shares, so honest parties will successfully verify shares from all honest dealers. Since the protocol repeats until no fraud proofs remain, eventually all remaining dealers will be honest or will have sent valid shares. After the protocol completes without fraud proofs, all honest parties will have received valid shares from all dealers and will output their secret shares.

To ensure at least t honest parties complete successfully, we need $n-f \geq t$, i.e., $n \geq t+f$. This gives $|\mathcal{S}| \geq n-f \geq t$ valid shares from honest parties. Note that when $f = t-1$ (worst case), this reduces to $n \geq 2t-1$. $\square$

Theorem 6 (Key Rotation Consistency and Correctness). *The key rotation protocol from Section 6.3 satisfies consistency and correctness (Definition 8), if we always have $n-f \geq t$ and $t \geq f+1$.*

Proof. Key Preservation: Each old dealer P_j^{old} sets $d_{j,0} = sk_j^{old}$ and the verification $D_{j,0} = \prod_{k=0}^{t-1} C_k^{old \cdot j^k}$ ensures dealers reshare their actual old shares. By Lagrange interpolation over $\mathcal{D}$ (with $|\mathcal{D}| = t$):

$$vk' = \prod_{j \in \mathcal{D}} D_{j,0}^{\lambda_{0,j,\mathcal{D}}} = g^{\sum_{j \in \mathcal{D}} \lambda_{0,j,\mathcal{D}} sk_j^{old}} = g^{sk} = vk$$

Correctness: Each new committee member P_i^{new} receives shares from t verified old dealers. The shares satisfy:

$$g^{s'_{j,i}} = \prod_{k=0}^{t'-1} D_{j,k}^{i^k}$$

ensuring they lie on degree- $(t'-1)$ polynomials. The reconstructed share is:

$$sk_i^{new} = \sum_{j \in \mathcal{D}} \lambda_{0,j,\mathcal{D}} \cdot s'_{j,i}$$

which correctly interpolates the polynomials. Since $|\mathcal{D}| = t$ and all are verified, we have $|\mathcal{S}'| \geq t'$ valid new shares for the same secret sk . $\square$

Theorem 7 (mEUF-CMA Security). *The distributed encrypted BLS scheme $(\Pi_{DKG}, \Pi_{Rot}, \mathit{ephkey}, \mathit{encsign}, \mathit{encverify}, \mathit{decrypt}, \mathit{verify})$ is mEUF-CMA secure (Definition 9). Specifically, for any PPT adversary $\mathcal{A}$ corrupting at most $t-1$ key servers initially and making at most q_{rot} key rotation queries, there exist PPT algorithms $\mathcal{S}, \mathcal{B}$ such that:*

$$\text{Adv}_{\mathcal{A}}^{mEUF}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{co-CDH}(\lambda) + (q_{rot} + 1) \cdot \text{Adv}_{\mathcal{S}}^{ADO-CCA}(\lambda)$$

where ADO-CCA is the Associated-Data-Only CCA security of the multi-recipient encryption scheme, and the factor $(q_{rot} + 1)$ accounts for the initial DKG plus all key rotations.

Proof. We prove security via a sequence of hybrid games, following the approach from the DKG literature.

Game 0: Real mEUF-CMA experiment where challenger emulates all honest parties through DKG and any key rotation queries.

Game 1: Challenger aborts if during DKG or any key rotation, there exists an adversarial party that submits an invalid fraud proof that is accepted.

Claim: $|\Pr[\text{Game 0}] - \Pr[\text{Game 1}]|$ is negligible.

Proof: This condition can only arise if the adversary posts a verifying non-empty fraud proof for an honest message. This contradicts the soundness of MREC.reveal.

Game 2: Challenger sets $s_{i,j} = 0$ when i, j are indexes of honest parties (encrypt zeros instead of real shares) during DKG and all key rotations, but still uses correct shares on the recipient side.

Claim: Games 1 and 2 are indistinguishable.

Proof: By ADO-CCA security of multi-recipient encryption. We construct adversary $\mathcal{D}$ against ADO-CCA:

- $\mathcal{D}$ receives honest parties' public keys
- For DKG: For each honest party P_i 's encryption, $\mathcal{D}$ queries encryption oracle with messages M_i^0 (real shares from Game 1) and M_i^1 (zeros from Game 2), $\text{tag} = i$
- For each key rotation query: Similarly encrypt zeros vs real reshares for honest parties
- When challenge bit is 0, $\mathcal{D}$ emulates Game 1; when 1, emulates Game 2

The reduction loses a factor of $(q_{rot} + 1)$ for the initial DKG plus q_{rot} rotations. Thus:

$$|\Pr[\text{Game 1}] - \Pr[\text{Game 2}]| \leq (q_{rot} + 1) \cdot \text{Adv}_{\mathcal{D}}^{\text{ADO-CCA}}(\lambda)$$

Game 3: Full simulator with co-CDH embedding. Challenger receives co-CDH challenge $(g_1, g_2, g_1^\alpha, g_1^\beta, g_2^\beta)$ and must compute $g_1^{\alpha\beta}$.

Simulation:

1. **DKG Setup:** For honest parties $i \in [t, n]$:
 - Sample $\theta_i, s_{i,1}, s_{i,2}, \dots, s_{i,t-1} \leftarrow \mathbb{Z}_q$. Set $s_{i,j} = 0$ for $j = t, \dots, n$.
 - Set $C_{i,0} = g_2^\beta \cdot g^{\theta_i}$ and compute $C_{i,k}$ corresponding to coefficients of X^k , $k \in [1, t-1]$ in polynomial p_i from coordinates $(0, C_{i,0}), (1, g_2^{s_{i,1}}), \dots, (t-1, g_2^{s_{i,t-1}})$.
 - Compute PoK π_i : To simulate the PoK without knowing β explicitly, we use the standard simulation technique for Schnorr proofs in the random oracle model. Sample $z_i, c_i \leftarrow \mathbb{Z}_q$ uniformly at random, then compute $R_i = g^{z_i} / C_{i,0}^{c_i}$. Since $C_{i,0} = g_2^\beta \cdot g^{\theta_i}$ and we assume $g_2 = g$ (same generator for DKG commitments), we have $C_{i,0} = g^{\beta+\theta_i}$, so $R_i = g^{z_i - c_i(\beta+\theta_i)}$. We can compute this using g_2^β from the co-CDH challenge (since $g_2 = g$) and known θ_i . Finally, program the random oracle to set $H(C_{i,0} \| R_i \| i) = c_i$. The proof is $\pi_i = (R_i, z_i)$, and it satisfies the verification equation $g^{z_i} = R_i \cdot C_{i,0}^{c_i}$ by construction.
 - Compute $E_i = \text{MREC.enc}(PK, S_i, i)$ where $S_i = (s_{i,1}, \dots, s_{i,n})$ using ADO-CCA encryption oracle.
 - Publish $(C_{i,0}, C_{i,1}, \dots, C_{i,t-1}, E_i, \pi_i)$.
2. **DKG Verification:** For each adversarial party $j \in [1, t-1]$, verify the PoK π_j and extract $c_{j,0}$. In the random oracle model, the simulator can extract $c_{j,0}$ from the PoK $\pi_j = (R_j, z_j)$ using the standard rewinding technique: obtain two valid proofs (R_j, z_j) and (R_j, z'_j) for different hash challenges c_j and c'_j (by rewinding the random oracle), then solve $c_{j,0} = (z_j - z'_j) / (c_j - c'_j)$ from the equations $g^{z_j} = R_j \cdot C_{j,0}^{c_j}$ and $g^{z'_j} = R_j \cdot C_{j,0}^{c'_j}$. Alternatively, the simulator can decrypt all

encrypted shares E_j using the decryption keys of honest parties (which the simulator controls), obtaining shares $s_{j,i}$ for all $i \in [n]$, then interpolate to recover the full polynomial $p_j(\cdot)$ and extract $c_{j,0} = p_j(0)$. The PoK provides a direct way to obtain $c_{j,0}$ without needing to decrypt and interpolate, which is particularly useful when $n - f < t$ (fewer than t honest parties available to decrypt shares). For honest parties, set $F_{i,j} \leftarrow \perp$. Complete DKG protocol. Let $\mathcal{J} = [t, n]$ (honest parties) and $\mathcal{J}' = [1, t - 1]$ (adversarial parties). Store the initial sets $\mathcal{J}_0 = \mathcal{J} = [t, n]$ and $\mathcal{J}'_0 = \mathcal{J}' = [1, t - 1]$ for use in secret key computation and forgery extraction.

3. **Key Rotation Simulation:** Upon receiving $\text{Rotate}(n', t', \mathcal{C}')$ query:

- Let honest new parties be $\mathcal{J}_{new} = [n'] \setminus \mathcal{C}'$ and adversarial new parties be $\mathcal{J}'_{new} = \mathcal{C}'$
- For each honest old party $i \in \mathcal{J}$ (acting as dealer):
 - Sample $s'_{i,1}, \dots, s'_{i,t'-1} \leftarrow \mathbb{Z}_q$. Set $s'_{i,j} = 0$ for $j = t', \dots, n'$.
 - Set $D_{i,0} = g_2^{sk_i}$ (computable from old VSS keys) and compute $D_{i,k}$ corresponding to coefficients of X^k , $k \in [1, t' - 1]$ in polynomial p_i from coordinates $(0, D_{i,0}), (1, g_2^{s'_{i,1}}), \dots, (t' - 1, g_2^{s'_{i,t'-1}})$. Note that since sk_i is a known affine function of β , $D_{i,k}$ can be computed from g_2^β .
 - Encrypt new shares $E'_i = \text{MREC.enc}(PK^{new}, S'_i, i)$ where $PK^{new} = \{\text{pk}_1^{new}, \dots, \text{pk}_{n'}^{new}\}$ and $S'_i = (s'_{i,1}, \dots, s'_{i,n'})$ using ADO-CCA encryption oracle.
 - Publish $(D_{i,0}, \dots, D_{i,t'-1}, E'_i)$
- Complete the rest of the key rotation protocol with the new committee members.
- Update $(\mathcal{J}, \mathcal{J}', n, t) \leftarrow (\mathcal{J}_{new}, \mathcal{J}'_{new}, n', t')$
- Recompute secret shares for new honest parties using Lagrange interpolation

4. **Secret Key Computation:** The joint key is $\text{vk} = (g_2^\beta)^{|\mathcal{J}_0|} \cdot g_2^{\sum_{j \in \mathcal{J}_0} \theta_j + \sum_{j \in \mathcal{J}'_0} c_{j,0}}$ where $\mathcal{J}_0 = [t, n]$ and $\mathcal{J}'_0 = [1, t - 1]$ are the initial honest and adversarial parties, and $c_{j,0}$ are the constant terms of adversarial polynomials from the initial DKG. The (implicit) secret key is $x = \beta \cdot |\mathcal{J}_0| + \sum_{j \in \mathcal{J}_0} \theta_j + \sum_{j \in \mathcal{J}'_0} c_{j,0}$. This key remains constant across rotations. Note that $|\mathcal{J}_0|$ and the sums over $\mathcal{J}_0$ and $\mathcal{J}'_0$ are fixed from the initial DKG, while $\mathcal{J}$ and $\mathcal{J}'$ refer to the current committee's honest and adversarial parties respectively.

For honest party $i \in \mathcal{J}$, the (implicit) secret share is:

$$sk_i = \sum_{j \in \mathcal{J}} p_j(i) + \sum_{j \in \mathcal{J}'} p_j(i)$$

where $p_j(i)$ denotes party j 's polynomial evaluated at i . To compute sk_i :

- For honest parties $j \in \mathcal{J}$: We know $p_j(0) = \beta + \theta_j$ and $p_j(k) = s_{j,k}$ for $k = 1, \dots, t - 1$. Use Lagrange interpolation with these implicit points to compute $p_j(i)$ as an affine function of β . Thus $g_1^{\sum_{j \in \mathcal{J}} p_j(i)}$ can be computed from g_1^β .
- For adversarial parties $j \in \mathcal{J}'$: We extract $c_{j,0}$ directly from the PoK π_j (as described in the DKG Verification step). To compute $p_j(i)$ for honest party i , we decrypt the encrypted share E_j (or E'_j during rotation) using honest receiver i 's decryption key to obtain $s_{j,i} = p_j(i)$ directly. We do not need to interpolate the full polynomial since we only need $p_j(i)$ for the specific honest party i whose share we are computing. The PoK allows extraction of $c_{j,0}$ even when $n - f < t$ (fewer than t honest parties available to decrypt shares).

At this point, we can write $sk_i = s_i + t_i \beta$, where s_i and t_i are efficiently computable values that depend on the recovered adversarial polynomials and the known honest party shares.

5. **RO Programming:** Sample $u \leftarrow \{1, \dots, q_H\}$. Set $H(ID_u) = g_1^\alpha$, $H(ID_i) = g_1^{r_i}$ for other queries.
6. **EphKey Simulation:** Return $\text{ephpk}_j = (g_1^{\beta+v_j}, g_2^{\beta+v_j})$ for random v_j . Store (ephpk_j, v_j) .

7. **EncSign Simulation:** For honest party $i \in \mathcal{J}$, compute sk_i as described in the Secret Key Computation step above, which gives $sk_i = s_i + t_i\beta$ where s_i and t_i are efficiently computable.

Then simulate encrypted signature shares:

- If $ID \neq ID_u$: Look up r_{ID} such that $H(ID) = g_1^{r_{ID}}$. Sample $w \leftarrow \mathbb{Z}_q$ and return $(g_1^w, \text{ephpk}_1^w \cdot g_1^{s_{k_i} \cdot r_{ID}})$.
- If $ID = ID_u$ and $\text{ephpk} \notin \mathcal{E}_k$: Abort (violates winning condition).
- If $ID = ID_u$ and $\text{ephpk} \in \mathcal{E}_k$: Look up v such that $\text{ephpk} = (g_1^{\beta+v}, g_2^{\beta+v})$. Sample $w \leftarrow \mathbb{Z}_q$ and return $(g_1^{w-\alpha \cdot t_i}, g_1^{(\beta+v)(w-\alpha \cdot t_i)+\alpha \cdot t_i \cdot \beta + \alpha \cdot s_i})$.

8. **Forgery Extraction:** If $\mathcal{A}$ outputs (ID^*, sig^*) where $e(sig^*, g_2) = e(H(ID^*), vk)$ and $ID^* = ID_u$, then $sig^* = H(ID_u)^x = (g_1^\alpha)^x$. Since $x = \beta \cdot |\mathcal{J}_0| + \sum_{j \in \mathcal{J}_0} \theta_j + \sum_{j \in \mathcal{J}'_0} c_{j,0}$ where $\mathcal{J}_0 = [t, n]$ and $\mathcal{J}'_0 = [1, t-1]$ are the initial honest and adversarial parties, compute $g_1^{\alpha\beta} = (sig^*)^{1/|\mathcal{J}_0|} \cdot (g_1^\alpha)^{-\sum_{j \in \mathcal{J}_0} \theta_j + \sum_{j \in \mathcal{J}'_0} c_{j,0}} / |\mathcal{J}_0|$.

Claim: Games 2 and 3 are distributed identically.

Proof: The commitments $C_{i,j}$ are distributed identically (can be computed from the given points without knowing the polynomial). The encrypted shares are zeros in both games. The partial signatures are valid in both games by the simulation strategy.

Conclusion: If $\mathcal{A}$ succeeds in Game 3, then challenger extracts co-CDH solution. The reduction loses factor q_H from guessing challenge ID. Combined with Game 2 reduction:

$$\text{Adv}_{\mathcal{A}}^{mEUF}(\lambda) \leq q_H \cdot \text{Adv}_{\mathcal{B}}^{co-CDH}(\lambda) + n \cdot \text{Adv}_{\mathcal{S}}^{ADO-CCA}(\lambda)$$

□

7 Multi-Recipient Encryption

The MPC committee of Section 6 distributes dealer shares with the multi-recipient scheme defined here. We use the Multi-Encryption Gadget (MEGa) of [GS22] which is a multi-recipient extension of the encryption of [SG98].

Definition 10 (Multi-Recipient Encryption). *A multi-recipient encryption scheme consists of five algorithms:*

- $\text{keygen}(1^\lambda) \rightarrow (\text{sk}, \text{pk})$: A probabilistic key generation algorithm that takes a security parameter 1^λ , and outputs a secret key, public key tuple (sk, pk) .
- $\text{enc}(\text{PK}, M, \text{tag}) \rightarrow E$: A probabilistic encryption algorithm that takes a public key sequence $\text{PK} = (\text{pk}_i)_{i=1}^n$, a sequence of plaintext messages $M = (m_i)_{i=1}^n$, and a tag tag as inputs and produces a ciphertext E (the identities of all parties are implicitly known).
- $\text{dec}(\text{sk}_i, i, E, \text{tag}) \rightarrow m$: A deterministic decryption algorithm that takes a secret key sk_i , an identity index i , a ciphertext E , and a tag tag as inputs and returns a plaintext message m , or a failure.
- $\text{reveal}(\text{sk}_i, i, E, \text{tag}) \rightarrow (m, \pi)$: A probabilistic reveal algorithm that takes a secret key sk_i , an index i , a ciphertext E , and a tag tag as inputs and returns a plaintext message m and a proof π . (This is a combination of the decryption and decryption proof algorithms of [GS22].)
- $\text{verify}(\text{pk}_i, m, \pi, i, E, \text{tag}) \rightarrow \{0, 1\}$: A deterministic algorithm that takes a public key pk_i , a plaintext m , a proof π , an index i , a ciphertext E , and a tag tag , and outputs a boolean value.

See [GS22] for security definitions. In a nutshell, we require that in an Associated-data-only CCA (ADO-CCA) game in which the adversary controls a subset of the parties and can choose their keys after seeing the honest parties' keys, an encryption with tag tag is secure even if the adversary can continually query reveal with $\text{tag}' \neq \text{tag}$ and with the honest parties' keys.

7.1 Construction

For simplicity, we describe the construction of [GS22] specifically for $\mathbb{G}_2$. Let H, H_2 be random oracles, instantiated using different domain separators in each invocation of the protocol. The construction is as following:

- **KeyGen** (per party): Sample secret key $sk \leftarrow \mathbb{Z}_q$. Publish $pk = g_2^{sk}$.
- **Encryption** $\text{enc}(PK, M, \text{tag})$, where PK denote the sequence $(pk_1, \dots, pk_n)$ and $M = (m_1, \dots, m_n)$:
 - Sample $r \leftarrow \mathbb{Z}_q$ and compute $c = g_2^r$.
 - Compute $\hat{g} = H_2(\text{tag}, c)$ where $\hat{g} \in \mathbb{G}_2$, and $\hat{c} = \hat{g}^r$.
 - Compute NIZK π_{ddh} that $(g_2, \hat{g}, c, \hat{c})$ is a DDH-tuple.
 - Compute $c_i \leftarrow H(\text{tag}, i, pk_i^r) \oplus m_i$ for $i = 1, \dots, n$.
 - Output the ciphertext $E := (c, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$.
- **Decryption** of the i -th message, $\text{dec}(sk_i, i, E, \text{tag})$:
 - Parse E as $(c, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$.
 - Verify π_{ddh} with respect to $c, \hat{c}$ and tag . Output $\perp$ in case of a failure.
 - Output $c_i \oplus H(\text{tag}, i, c^{sk_i})$.
- **Reveal** of the i -th message, $\text{reveal}(sk_i, i, E, \text{tag})$:
 - Parse E as $(c, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$.
 - Verify π_{ddh} with respect to $c, \hat{c}$ and tag . Output $\perp$ if in case of a failure.
 - Let $m = c_i \oplus H(\text{tag}, i, c^{sk_i})$.
 - Let π' be a NIZK that $(g_2, c = g_2^r, pk_i = g_2^{sk_i}, c^{sk_i})$, with witness sk_i .
 - Output $(m, (c^{sk_i}, \pi'))$.
- **Verify** $\text{verify}(pk_i, m, \pi, i, E, \text{tag})$:
 - Check that $pk_i \in \mathbb{G}_2 - \{1_{\mathbb{G}}\}$.
 - Parse π as (c', π') .
 - Parse E as $(c, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$.
 - Verify proof π' with (g_2, c, pk_i, c') and if $m = c_i \oplus H(\text{tag}, i, c')$.

The security of the above construction can be reduced to aCDH in $\mathbb{G}_2$, i.e., given $g_1, g_2, g_1^x, g_2^x, g_1^y, g_2^y$ compute g_2^{xy} , and the simulation soundness and ZK of the NIZKs. The hybrid games are the following:

Game 0. The real execution where the simulator emulates all honest parties.

Game 1. The simulator samples $y \leftarrow \mathbb{Z}_q$ and sets $pk_i = g_2^{y+y_i}$ where y_i is sampled at random and i is an index of an honest party.

Games 0 and 1 are indistinguishable as the keys are distributed exactly the same.

Game 2. The simulator programs the random oracle H_2 to return $g_2^{y+z_j}$ for a random z_j on the j -th new input query done by the adversary.

Game 2 is indistinguishable from Game 1 since the output of the random oracle are distributed exactly the same in both games (unless the same input was queried earlier by the simulator, but that should happen with a negligible probability).

Game 3. Replace the NIZKs generated by honest parties with simulated proofs.

By the ZK of those proofs, this game is indistinguishable from Game 2.

Game 4. Recall that for a ciphertext $E = (c = g_2^r, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$ created by the adversary, the expected DH key for the i -th honest party is $pk_i^r = g_2^{(y+y_i)r}$. Observe that after Game 2, we know

that: if π_{ddh} is valid with respect to $c = g_2^r$, $\hat{c} = \hat{g}^r$, then $\hat{c} = \hat{g}^r = g_2^{(y+z_j)r}$, thus the simulator can compute $\text{pk}_i^r = \hat{c} \cdot c^{y_i - z_j}$ with the relevant z_j .

Now, on a **Reveal** query, the simulator computes the DH value pk_i^r as described above instead of using the private keys.

Games 3 and 4 are computationally indistinguishable following the simulation soundness of the NIZK π_{ddh} .

Game 5. The simulator samples $x \leftarrow \mathbb{Z}_q$. When the j -th encryption query with **tag** is requested, the simulator returns the ciphertext $E = (c, \hat{c}, \pi_{\text{ddh}}, c_1, \dots, c_n)$ where $c = g_2^x \cdot g_2^{x_j}$ for a random x_j , $\hat{c} = c^z$ where z is random, and c_i -s are random strings. It also programs the random oracle H_2 to return g_2^z when queried with c .

When the adversary queries $H(\text{tag}, i, q)$ for i belonging to the adversary, if $e(g_1^x \cdot g_1^{x_j}, \text{pk}_i) = e(g_1, q)$ then it returns the value $c_i \oplus m_i$ (where m_i is the j -th challenge's, i -th message; backpatching the oracle to return the correct decrypted value).

When the adversary queries $H(\text{tag}, i, q)$ for i belonging to an honest party, the simulator checks the same condition as above. If that is the case, it aborts.

Games 4 and 5 are the same except for when the simulator aborts in the last step. We argue that the abort probability is negligible following aCDH: Consider Game 5' that is the same as Game 5, but where the simulator receives the aCDH challenge g_1^x, g_2^x, g_2^y and uses those values in the simulation above. Games 5 and 5' are distributed the same. Now, if the simulator aborts in Game 5', it means that it learns $q = g_2^{(x+x_j)(y+y_i)}$, thus the simulator wins the aCDH game by outputting $q / ((g_2^y)^{x_j} (g_2^x)^{y_i} g_2^{x_j y_i})$. By the security of aCDH, the probability of such an abort in Game 5' is negligible, and so is the one in Game 5.

Observe that in Game 5 the adversary does not have any advantage in guessing the challenge bit as the honest parties' encryptions are random strings. We conclude that since Game 5 and the real execution are indistinguishable, the adversary advantage in guessing the challenge bit is negligible in the real execution as well.

Correctness and Completeness [GS22] follow the construction. Soundness, meaning that the adversary cannot fake a Reveal response for a correct encryption, follows the soundness of the NIZK.

References

- [BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229, Santa Barbara, CA, USA, August 19–23, 2001. Springer Berlin Heidelberg, Germany.
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 514–532, Gold Coast, Australia, December 9–13, 2001. Springer Berlin Heidelberg, Germany.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press.
- [CCN⁺23] Andrea Cerulli, Aisling Connolly, Gregory Neven, Franz-Stefan Preiss, and Victor Shoup. vetKeys: How a blockchain can keep many secrets. *Cryptology ePrint Archive*, Report 2023/616, 2023.
- [GS22] Jens Groth and Victor Shoup. Design and analysis of a distributed ECDSA signing service. *Cryptology ePrint Archive*, Report 2022/506, 2022.
- [LQ05] Benoît Libert and Jean-Jacques Quisquater. Identity based encryption without redundancy. In John Ioannidis, Angelos Keromytis, and Moti Yung, editors, *ACNS 2005: 3rd International Conference on Applied Cryptography and Network Security*, volume 3531 of *Lecture Notes in Computer Science*, pages 285–300, New York, NY, USA, June 7–10, 2005. Springer Berlin Heidelberg, Germany.

- [SG98] Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In Kaisa Nyberg, editor, *Advances in Cryptology – EUROCRYPT’98*, volume 1403 of *Lecture Notes in Computer Science*, pages 1–16, Espoo, Finland, May 31 – June 4, 1998. Springer Berlin Heidelberg, Germany.